

Improving Dynamic Programming for the Quadratic Knapsack Problem

by

Thijs Niekerk (SNR: 2125613)

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor in Econometrics and Operations Research

Tilburg School of Economics and Management
Tilburg University

Supervised by: Ties Schalij

Date: June 11, 2026

Abstract

The Quadratic Knapsack Problem (QKP) is a strongly NP-hard combinatorial optimization problem that extends the classical knapsack problem with pairwise interaction terms between items. The Fomeni-Letchford (F&L) dynamic programming heuristic is a well-established approach for the QKP, but its static item ordering and greedy single-path selection cause it to perform poorly on structured instances. This thesis formally proves that these weaknesses result in an unbounded approximation ratio and proposes targeted improvements to restore effective performance.

We introduce two principal enhancements: a diversity-preserving beam search that maintains multiple structurally distinct candidate solutions per state, and a dynamic item ordering strategy that adaptively resequences items based on realized synergies. We show that both improvements individually resolve the unbounded approximation ratio of the F&L heuristic, provided that the beam width is sufficiently large, and that they are complementary in practice. Furthermore, empirical results demonstrate that our combined approach is competitive with or better than the Fennich et al. (2024) heuristic in overall solution quality, while running substantially faster on large instances.

Contents

1 Introduction **4**
- 1.1 Classical Knapsack Problem . . . 4
- 1.2 Quadratic Knapsack Problem . . . 4
- 1.3 Research Question and Approach . . . 6
- 1.4 Thesis Outline . . . 7

2 Preliminaries **8**
- 2.1 Dynamic Programming Fundamentals . . . 8
- 2.2 Dynamic Programming for Classical Knapsack . . . 9

3 Quadratic Knapsack Problem **11**
- 3.1 Problem Formulation . . . 11
- 3.2 The F&L Dynamic Programming Heuristic . . . 12
  - 3.2.1 Complexity Analysis . . . 15
- 3.3 Limitations of the F&L Heuristic . . . 15
  - 3.3.1 Static Ordering . . . 15
  - 3.3.2 Greedy Selection in the DP . . . 16
- 3.4 Unbounded Approximation Ratio . . . 16
- 3.5 Addressing F&L Limitations: The Approach by Fennich et al. (2024) . . . 19
  - 3.5.1 Computational Complexity of the Fennich Approach . . . 21

4 Proposed Improvements **22**
- 4.1 Beam Search . . . 22
  - 4.1.1 Multi-Solution Local Search . . . 23
  - 4.1.2 Diverse Beam Search . . . 24
  - 4.1.3 Complexity . . . 25
  - 4.1.4 Beam Width Selection . . . 26
- 4.2 Dynamic Ordering . . . 26
  - 4.2.1 Algorithm . . . 26
  - 4.2.2 Complexity . . . 28
- 4.3 The Combined Approach . . . 28
  - 4.3.1 Algorithmic Integration . . . 29
  - 4.3.2 Complexity Analysis . . . 29
- 4.4 Summary of Improvements . . . 30

5 Experimental Results **31**
5.1 Experimental Setup . . . 31
5.2 Instance Classes . . . 32
5.2.1 Gallo Instances . . . 32
5.2.2 Densest k-Subgraph (DKS) Instances . . . 32
5.2.3 Hidden Clique Instances . . . 33
5.3 Beam Width Experiments . . . 33
5.3.1 Diverse Beam vs. Standard Beam . . . 33
5.3.2 Standard Instances . . . 34
5.3.3 Hidden Clique Instances . . . 37
5.3.4 Analytical Derivation of the Non-Monotone Difficulty Profile . . . 41
5.4 Comparisons . . . 43
5.4.1 Gallo Instances . . . 44
5.4.2 Densest k-Subgraph Instances . . . 46
5.4.3 Hidden Clique Instances . . . 48
5.5 Validation of the Fennich baseline . . . 50
5.6 Local Search Improvements . . . 52
5.7 Algorithmic Refinement . . . 53
5.7.1 Aggregated Anchor Approach . . . 53
5.7.2 Sensitivity Analysis . . . 54
5.7.3 Full Evaluation . . . 54

6 Conclusion **57**

A Extra Figures **63**

Chapter 1

Introduction

1.1 Classical Knapsack Problem

The 0-1 knapsack problem, also known as the rucksack problem, is well known in combinatorial optimization (Dantzig, 1957). The problem can be best explained by an example. Consider a thief who has a rucksack with a weight capacity c and wants to steal a selection from n available items that will maximize the total value. Every item i has a value q_i and a weight w_i, so the thief needs to steal a number of items where the sum of the w_i is smaller than or equal to c and where the sum of the q_i is maximized (Cormen et al., 2009). This problem is theoretically significant because it is one of Karp's (1972) original NP-complete problems, acting as a benchmark for testing the efficiency of new optimization algorithms. Practically, the problem is essential for modeling situations where decision makers must allocate limited resources among competing choices to maximize total utility (Cacchiani et al., 2022a). Because this problem is so versatile and standard, it has been studied extensively in the literature (Salkin and de Kluyver, 1975), with solution approaches ranging from exact algorithms that guarantee optimal solutions to heuristic methods that trade optimality for computational efficiency. The most well-known exact approach is dynamic programming (DP), which solves the knapsack problem in $O(nc)$ time, where n is the number of items and c is the capacity (Bellman, 1957). This pseudo-polynomial complexity makes DP highly effective for instances with moderate capacities. Branch-and-bound algorithms provide another exact approach, using upper bounds to prune the search tree, reducing computational effort (Kolesar, 1967).

1.2 Quadratic Knapsack Problem

The Quadratic Knapsack Problem (QKP) is an extension of the classical 0-1 knapsack problem, having been formally defined by Gallo, Hammer, and Simeone in 1980 (Gallo et al., 1980). While the classical version considers only the individual profit of each item, the QKP introduces interaction terms, q_{ij}, which represent the added value gained when both items i and j are selected. These terms can represent bonuses (positive interaction) or conflicts (negative

interaction), and they are only activated when both item i and item j are included in the chosen set.

A foundational application of the QKP is the optimization of satellite ground station networks (Caprara et al., 1999). In this scenario, the items represent potential geographical locations for the stations, while the capacity represents the maximum available budget for infrastructure development. The primary objective is to maximize global communication traffic across the network. Because data transmission is an inherently pairwise activity, the utility of the system is derived from the links between stations rather than the stations themselves. This provides a perfect realization of the quadratic interaction terms q_{ij}, as the profit is only unlocked when both station i and station j are selected for the set.

A naive dynamic programming formulation would require state space $O(n \cdot c \cdot 2^n)$ to track both capacity and the subset of selected items, resulting in exponential complexity. This computational barrier has motivated the development of various approximate and heuristic approaches, as well as exact methods that exploit problem structure to remain solvable for problems with a moderate c.

Research on the Quadratic Knapsack Problem has explored both exact methods and heuristic approaches. We provide a brief overview of the main approaches, organizing them by solution methodology (Cacchiani et al., 2022b).

Several exact algorithms based on branch-and-bound have been developed. Billionnet and Soutif (2004a) presented an algorithm using Lagrangian decomposition to compute upper bounds, solving instances with up to $n \in [150, 300]$ variables depending on profit matrix density. Pisinger et al. (2007) developed an algorithm employing aggressive reduction combined with multiple upper bounds, solving instances with up to 1500 variables. More recently, Fomeni et al. (2022) presented a cut-and-branch algorithm incorporating cutting planes, primal heuristics, and reduction rules, demonstrating strong performance on challenging instance classes and solving standard instances with up to 800 variables within a time span of 5 hours. Complementing these are linearization methods that transform the quadratic objective into an equivalent mixed-integer program. Billionnet and Soutif (2004b) presented one existing linearization method and proposed two original approaches of their own design. On the other hand, Rodrigues et al. (2012) took a different approach by turning the quadratic terms into several linear constraints. They combined this with a branch-and-bound algorithm, which worked especially well for low-density problems with as many as 400 variables.

Fomeni and Letchford (2014) showed that the classical DP framework for the linear knapsack can be adapted to obtain an effective heuristic for QKP. Their approach maintains the same state space $\mathrm{DP}[i, w]$ as in the classical algorithm, avoiding the exponential complexity that would arise from explicitly tracking selected item subsets. The key idea is to use the standard DP recursion but evaluate the quadratic objective heuristically through intelligent item ordering and tie-breaking rules. While this heuristic does not guarantee optimal solutions because it cannot exactly compute quadratic interaction terms without tracking which items are selected, it achieves remarkably good perfor-

mance. The approach was successfully compared with other heuristic methods, demonstrating competitive solution quality (Cunha et al., 2016; Chen and Hao, 2017). The effectiveness of this DP heuristic, despite its approximations, suggests that under certain problem structures, the loss of information from not tracking the complete item subset may have minimal impact on solution quality. This observation motivates an investigation into conditions under which the DP framework could be modified to provide exact solutions.

While the Quadratic Knapsack Problem has received considerable attention in the optimization literature, significant questions remain regarding the performance and potential improvements of dynamic programming-based approaches on challenging instances. Fomeni and Letchford's DP heuristic demonstrated impressive performance on standard benchmark instances, achieving solution quality within 0.05% of optimality when combined with local search refinement. However, these benchmarks follow the instance generation scheme of Gallo et al. (1980), where profits are independent random variables, and the knapsack capacity is proportional to the number of items. Schauer (2016) demonstrated both theoretically and empirically that such instances possess a particular structure that makes them asymptotically easy to solve, meaning that for this class of instances, the ratio between the optimal solution value and that obtained by a trivial greedy heuristic converges to one as the instance size grows.

To provide more challenging benchmarks, Schauer (2016) introduced hidden clique instances based on random graphs. These instances violate the independence assumptions underlying the asymptotic analysis, as the quadratic profit structure is determined by a planted clique within a random graph. Schauer's computational experiments revealed that the Fomeni-Letchford heuristic experiences a significant drop in performance on these harder instances, with optimality gaps increasing from near-zero on standard benchmarks to over 20% for larger hidden clique instances. The drop in performance highlights fundamental design weaknesses, and identifying and addressing these is the subject of this thesis.

1.3 Research Question and Approach

Motivated by the performance drop on complex instances identified above, this thesis investigates whether targeted enhancements to the dynamic programming framework can restore effective performance on challenging QKP instances. Specifically, we address the following research question: To what extent can the Fomeni and Letchford heuristic be improved on instances with complex quadratic interactions by incorporating dynamic item ordering and beam search enhancements?

Our approach addresses two fundamental limitations of the baseline heuristic: its static item ordering and its greedy state selection, by introducing concrete algorithmic modifications. To address the static ordering limitation, we develop a dynamic ordering strategy that recalculates item priorities at each stage of the DP algorithm based on which items the current best partial solu-

tion contains. This allows the algorithm to recognize strong item connections as it builds the solution and adapts its choices to take advantage of them. To address the greedy selection limitation, we implement a beam search variant, inspired by Ow and Morton (1988), that maintains the top-B highest-value solutions at each state rather than committing to a single choice. This creates multiple exploration paths through the solution space, reducing vulnerability to early suboptimal decisions. We implement these enhancements both individually and in combination, enabling systematic evaluation of their separate and joint effects.

1.4 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 introduces the foundations of dynamic programming and the classic knapsack algorithm. In Chapter 3, we define the Quadratic Knapsack Problem (QKP) and analyze the baseline Fomeni-Letchford (F&L) heuristic. This chapter also examines the structural flaws of the F&L approach, including its unbounded approximation ratio, and reviews a recent method by Fennich et al. (2024). Chapter 4 describes our core contributions. This includes the beam search variant, the dynamic ordering strategy, and their integration alongside complexity analyses. We evaluate these methods in Chapter 5 by benchmarking them against baselines across all three instance classes: standard Gallo, Densest k-Subgraph, and hidden clique instances. Finally, Chapter 6 summarizes the findings, discusses research limitations, and suggests future work.

Chapter 2

Preliminaries

2.1 Dynamic Programming Fundamentals

The knapsack problem is classified as NP-hard, which Karp established through a reduction from the exact cover problem (Karp, 1972). This means that no polynomial-time algorithm is known that guarantees optimal solutions for all instances. Despite this theoretical hardness, the knapsack problem can be solved in pseudo-polynomial time $O(nc)$ using dynamic programming (Bellman, 1957). An algorithm is pseudo-polynomial if its running time is polynomial in the numerical value of the input but exponential in the size of the input encoding (Garey and Johnson, 1979). While $O(nc)$ appears polynomial in n and c, the input size is $O(n \log_2 c + n \log_2 W)$, where W is the largest value among all items (Sotirov, 2025). Since each integer requires logarithmic space to represent in binary (Peeters, 2025), the complexity is exponential in the number of bits needed to encode the input. For example, if $c = 2^{20}$, only 20 bits are needed to represent c, but the algorithm performs approximately $n \cdot 2^{20}$ operations, which is exponential in the input length. Nevertheless, this pseudo-polynomial complexity has significant practical implications. For many real-world applications where the capacity c is not excessively large, the dynamic programming approach remains highly efficient. This explains why the knapsack problem, despite being NP-hard, can often be solved to optimality in practice. This motivates a closer examination of dynamic programming as a method for solving optimization problems.

Dynamic programming, introduced by Bellman in the 1950s at the RAND Corporation (Bellman, 1957), is a problem-solving approach for optimization problems across diverse fields, including spacecraft trajectory planning, inventory management, and resource allocation (Bellman, 1966). Interestingly, the term 'programming' here refers to a tabular method rather than computer programming (Cormen et al., 2009).

A problem can be effectively solved using dynamic programming only when it possesses two fundamental properties: optimal substructure and overlapping subproblems. First, a problem possesses **optimal substructure** if an optimal solution to the overall problem contains optimal solutions to its smaller subproblems. This implies that a global optimum can be efficiently constructed

from the solutions of smaller instances. Secondly, the problem must involve **overlapping subproblems**, which occur when a recursive algorithm revisits the same subproblems repeatedly rather than always generating new ones. Instead of recomputing these results, dynamic programming stores the solutions to subproblems in a table, thereby reducing the computational complexity from exponential to polynomial time (Cormen et al., 2009). In Section 2.2, we will formally demonstrate that the 0/1 knapsack problem possesses both of these essential properties, making dynamic programming an effective solution approach.

The theoretical foundation for dynamic programming is the principle of optimality. Bellman formulates it as the following:

> An optimal policy has the property that, whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. (Bellman, 1966, p. 35)

In essence, this principle states that an optimal solution can be broken down into optimal sub-solutions. It implies that if an initial optimal decision leads to a specific state, all future decisions from that state must remain optimal to ensure the global result is also optimal. Crucially, the optimality of these future decisions depends entirely on the current state rather than the specific sequence of choices made to reach it.

2.2 Dynamic Programming for Classical Knapsack

The 0/1 knapsack problem can be formally defined as follows. Given a set of n items, each item i (for $i = 1, 2, \ldots, n$) is associated with a positive integer weight $w_i \in \mathbb{Z}^+$ and a positive integer value $q_i \in \mathbb{Z}^+$. Additionally, there is a knapsack with a maximum weight capacity $c \in \mathbb{Z}^+$. The objective is to determine values for the binary decision variables x_i, where $x_i = 1$ if item i is selected and $x_i = 0$ otherwise, to maximize the total value:

$$\begin{aligned} \underset{x}{\text{maximize}} \quad & \sum_{i=1}^{n} q_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{n} w_i x_i \leq c, \\ & x_i \in \{0, 1\}, \quad \forall i \in \{1, \ldots, n\} \end{aligned}$$

The 0/1 knapsack refers specifically to these binary decisions, meaning that each item can be included at most once in the knapsack (Dantzig, 1957).

Having established that dynamic programming applies to problems with optimal substructure and overlapping subproblems, we now demonstrate that the 0/1 knapsack problem possesses both properties.

The knapsack problem possesses optimal substructure: for any item i, an optimal solution either includes it or not. If included, the remaining items must be optimally selected for capacity $c-w_i$; if excluded, for the full capacity c. The global optimum is thus built from optimal solutions to smaller subproblems.

It also exhibits overlapping subproblems. Consider deciding whether to include item 4 from $\{1, 2, 3, 4\}$ because the recursion splits into two branches, each of which splits again when considering item 3, and so on. Since subproblems are defined only by the remaining items and remaining capacity, different branches inevitably arrive at identical states. For example, the optimal solution for items $\{1, 2\}$ at a given capacity may be required by multiple branches. A naive recursive approach recomputes these states every time, resulting in $O(2^n)$ complexity. Storing solutions in a table ensures each state is computed only once, reducing this to $O(nc)$ (Cormen et al., 2009).

We define $f[i, w]$ as the maximum value achievable using items $\{1, \ldots, i\}$ with capacity w, where $0 \leq i \leq n$ and $0 \leq w \leq c$. The base cases are:

$$f[0, w] = 0 \text{ for all } w, \quad \text{and} \quad f[i, 0] = 0 \text{ for all } i \tag{2.1}$$

For $i > 0$ and $w > 0$, we have two options for each item i:

1. Exclude item i: The maximum value is $f[i-1, w]$, the best solution using items $\{1, \ldots, i-1\}$ with capacity w.

2. Include item i (if $w_i \leq w$): We gain value q_i and have remaining capacity $w - w_i$, giving value $q_i + f[i-1, w-w_i]$.

Taking the maximum of these two options gives the following recurrence relation:

$$f[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ f[i-1, w] & \text{if } w_i > w \\ \max(f[i-1, w],\ q_i + f[i-1, w-w_i]) & \text{if } w_i \leq w \end{cases} \tag{2.2}$$

This recurrence relation captures the principle of optimality: at each stage, we make the locally optimal decision knowing that the remaining decisions will also be optimal (Peeters, 2025; Bellman, 1966).

Chapter 3

Quadratic Knapsack Problem

3.1 Problem Formulation

The Quadratic Knapsack Problem (QKP) extends the classical version by representing profits through a symmetric matrix $Q \in \mathbb{Z}^{n \times n}$, where q_{ii} integrates the individual profit of item i and q_{ij} (for $i \neq j$) represents the joint profit of selecting both items i and j, also referred to as synergies. In this symmetric formulation, the pairwise interaction is captured by both q_{ij} and q_{ji}, where $q_{ij} = q_{ji}$. The problem is formulated as follows:

$$\begin{aligned} \underset{x}{\text{maximize}} \quad & \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} q_{ij} x_i x_j \\ \text{subject to} \quad & \sum_{i=1}^{n} w_i x_i \leq c, \\ & x_i \in \{0, 1\}, \quad \forall i \in \{1, \ldots, n\} \end{aligned}$$

In this formulation, the objective function sums all entries q_{ij} for which both x_i and x_j are equal to 1. Because $x_i^2 = x_i$ for binary variables, the linear terms are naturally incorporated as the diagonal elements q_{ii} of the matrix Q. Note that under this symmetric formulation, the diagonal entry q_{ii} appears only once in the double sum, contributing $\frac{1}{2} q_{ii}$ to the objective. To ensure the effective linear contribution equals q_i, we define $q_{ii} = 2q_i$ throughout this thesis. This symmetric approach is adopted to maintain consistency with the quadratic form $\frac{1}{2} x^\top Q x$ used in the computational implementation. While Q may in general contain negative entries representing conflicts between items, all experimental instances in this thesis use a non-negative profit matrix $Q \geq 0$.

While the standard 0-1 knapsack problem is known to be NP-hard (Karp, 1972), it is only weakly NP-hard because it can be solved in pseudo-polynomial time using dynamic programming. However, the addition of quadratic interaction terms makes QKP strongly NP-hard. This strong NP-hardness follows from the fact that the Maximum Clique problem, which is well-known to be strongly NP-hard (Karp, 1972; Caprara et al., 1999), can be reduced to QKP, a relationship first modeled by Gallo et al. (1980). By setting the item weights to $w_j = 1$, the linear profits to $q_{ii} = 0$, the capacity to $c = k$, and the quadratic

profits to $q_{ij} = 1$ if an edge exists between nodes i and j, any graph has a clique of size k if and only if the optimal QKP solution equals $\frac{1}{2}k(k-1)$. Therefore, QKP directly inherits the strong NP-hardness of the clique problem.

Unlike linear knapsack where each item has fixed profit q_i, in QKP an item's contribution depends on which other items are selected. Item i contributes $q_i + \frac{1}{2}(2\sum_{j \in S} q_{ij})$ when added to solution set S. An item that appears inefficient in isolation may become highly valuable when combined with complementary items, and vice versa. This context dependency fundamentally distinguishes QKP from linear optimization problems and complicates the construction of efficient solution methods (Fomeni and Letchford, 2014).

The linear knapsack objective $\sum_i q_i x_i$ is separable, meaning each term depends on a single variable. The quadratic objective $\frac{1}{2}(\sum_{i,j} q_{ij} x_i x_j)$ is fundamentally non-separable: selecting items i and j together yields profit $q_i + q_j + \frac{1}{2}(2q_{ij})$, which cannot be decomposed into independent contributions. This non-separability prevents decomposition techniques that work well for linear problems (Fomeni and Letchford, 2014).

Schauer (2016) demonstrated that QKP exhibits extreme variability in practical difficulty. Standard randomly generated instances following Gallo et al. (1980) prove asymptotically easy, where many algorithms quickly find optimal or near-optimal solutions. However, specially constructed instances based on hidden clique structures remain extremely challenging, with sophisticated algorithms producing solutions far from optimal. This variability reflects the complex variation between instance structure and algorithmic approaches.

Due to its strong NP-hardness and computational burden, the QKP is typically addressed through heuristic approaches that sacrifice optimality guarantees for computational efficiency. The following chapter examines how dynamic programming principles can be adapted to provide such heuristic methods.

3.2 The F&L Dynamic Programming Heuristic

As established in Section 2.1, dynamic programming achieves polynomial complexity when a problem exhibits optimal substructure and overlapping subproblems.

The quadratic knapsack problem fundamentally violates the optimal substructure property. Because profits include interaction terms q_{ij} between pairs of items, the value of the remaining subproblem now does depend on which specific items have already been selected. To illustrate this violation, consider three items with weights $w_1 = w_2 = w_3 = 1$, a capacity $c = 2$, and a symmetric profit matrix Q. The diagonal elements (representing linear profits) are $q_1 = q_2 = q_3 = 10$. We define the interaction between items 1 and 3 as $q_{13} = q_{31} = 100$, while all other off-diagonal elements are zero ($q_{12} = q_{21} = q_{23} = q_{32} = 0$). Suppose we are at a stage where we must choose one item from $\{2, 3\}$ with a remaining capacity of 1. **Case 1:** If item 1 was already selected, item 3 is clearly superior. The total profit for selecting items

$\{1, 3\}$ is $q_1 + q_3 + \frac{1}{2}(q_{13} + q_{31}) = 120$. In contrast, selecting items $\{1, 2\}$ results in a total profit of $q_1 + q_2 + \frac{1}{2}(q_{12} + q_{21}) = 20$. **Case 2:** If item 1 was not selected, both items 2 and 3 yield an identical profit of 10 (q_2 or q_3, respectively). The optimal decision for the subproblem (choosing between 2 and 3) changes based on the previous decision regarding item 1. This dependency violates the principle of optimality, as the best choice for a state depends on the path taken to reach it. Consequently, the recursive structure of standard dynamic programming cannot be directly applied, as the state representation would need to include the entire set of selected items to calculate future synergies, leading to an exponential size of the state space.

Fomeni and Letchford (F&L) (2014) proposed a polynomial-time dynamic programming heuristic for the QKP. Their approach adapts the classical knapsack Dynamic Programming (DP) framework while making strategic approximations to handle the quadratic objective function. As established in the example above, the quadratic interactions in the QKP break the optimal substructure required for standard dynamic programming. This violation forces a fundamental change in the state space meaning we can no longer index subproblems by (i, w) alone. Instead, to calculate an exact solution, we must track the complete set $S \subseteq \{1, \ldots, i\}$ of items selected so far to account for all future synergies. This causes the state space to expand to $f[i, w, S]$, containing an exponential $O(nc \cdot 2^n)$ states leading to exponential time complexity. The F&L heuristic addresses this by making three design choices:

1. **Static item ordering:** Compute a greedy priority ranking at initialization
2. **Greedy state selection:** Maintain only one solution per (i, w) state
3. **Local search refinement:** Apply a fill-up-and-exchange procedure post-DP to improve solution quality

We now describe each component of the algorithm in detail.

The first step in the algorithm is the static item ordering. Rather than processing items in their natural order $1, 2, \ldots, n$, the F&L algorithm reorders items according to a greedy priority metric. While the original authors evaluated several upper bounds, they specifically utilized a metric denoted as $\tilde{\pi}_3$ (Fomeni and Letchford, 2014) for their ordering phase, identifying it as the most effective for their heuristic. For the sake of notational simplicity in this thesis, we refer to this specific upper bound simply as $\tilde{\pi}$. For each item i, we define $\tilde{\pi}$ as:

$$\tilde{\pi}_i = q_{ii} + \max \left\{ \sum_{j \neq i} q_{ij} x_j : \sum_{j \neq i} w_j x_j \leq c - w_i, \quad 0 \leq x_j \leq 1, \quad j \neq i \right\} \tag{3.1}$$

This quantity represents the maximum possible profit contribution from item i: its individual profit q_i plus all possible pairwise interactions with other items. Items are sorted in non-increasing order of the efficiency ratio $\tilde{\pi}_i / w_i$. Let $\sigma : \{1, \ldots, n\} \to \{1, \ldots, n\}$ denote this ordering, such that:

$$\frac{\tilde{\pi}_{\sigma(1)}}{w_{\sigma(1)}} \geq \frac{\tilde{\pi}_{\sigma(2)}}{w_{\sigma(2)}} \geq \cdots \geq \frac{\tilde{\pi}_{\sigma(n)}}{w_{\sigma(n)}} \tag{3.2}$$

The DP then processes items in this order: $\sigma(1), \sigma(2), \ldots, \sigma(n)$. This ordering remains fixed throughout the algorithm, so we call it static ordering. Items with high profit potential per unit weight are considered first, following the greedy intuition that these are likely to be valuable.

Next in the algorithm is the DP recurrence phase, where greedy state selection is used. The F&L heuristic maintains a DP table with states indexed by (i, w), where $i \in \{0, \ldots, n\}$ represents the number of items processed (in the ordering σ) and $w \in \{0, \ldots, c\}$ represents the total weight used. Let $f[i, w]$ be the best profit found using items $\sigma(1), \ldots, \sigma(i)$ with total weight w. For each state, we store:

$$S[i, w] := \text{set of items achieving profit } f[i, w] \tag{3.3}$$

Base cases are $f[0, 0] = 0$ with $S[0, 0] = \emptyset$, and $f[0, w] = -\infty$ for $w > 0$.

At each subsequent state (i, w), the algorithm evaluates the transition from the previous stage $i - 1$ using the following recurrence:

$$f[i, w] = \max \begin{cases} f[i-1, w] & \text{(Exclude } \sigma(i)\text{)} \\ f[i-1, w - w_{\sigma(i)}] + q_{\sigma(i)} + \frac{1}{2}\left(2 \sum_{j \in S[i-1, w-w_{\sigma(i)}]} q_{\sigma(i),j}\right) & \text{(Include } \sigma(i)\text{)} \end{cases} \tag{3.4}$$

The associated item set $S[i, w]$ is updated according to the selection made in (3.4):

$$S[i, w] = \begin{cases} S[i-1, w] & \text{if item } \sigma(i) \text{ is excluded} \\ S[i-1, w - w_{\sigma(i)}] \cup \{\sigma(i)\} & \text{if item } \sigma(i) \text{ is included} \end{cases} \tag{3.5}$$

This selection represents the critical approximation of the heuristic: among all possible subsets of $\{\sigma(1), \ldots, \sigma(i)\}$ that sum to exactly weight w, only the one yielding the maximum instant profit is retained. In the event of a tie in profit, the algorithm prioritizes the solution with the higher cardinality:

$$S[i, w] = \arg\max \{|S| : S \text{ achieves } f[i, w]\} \tag{3.6}$$

All other configurations which might have offered superior synergy with future items $\sigma(i+1), \ldots, \sigma(n)$ are permanently discarded. The final solution is given by $\max_{w \leq c} f[n, w]$.

After the DP recurrence phase completes, F&L applies a local search improvement procedure called "fill-up-and-exchange" (Fomeni and Letchford, 2014). Starting from the solution S produced by the DP, the algorithm first performs a greedy fill-up phase, where any items $j \notin S$ that fit within the remaining capacity are added to the set. Subsequently, the exchange phase iteratively considers 1-exchange moves:

1. Remove one item $i \in S$
2. Add one item $j \notin S$, given that the swap is feasible (total weight $\leq c$)

For a given item i removed in Step 1, the algorithm continuously iterates through potential replacement items j in Step 2. As soon as a feasible swap improves the objective value, the algorithm uses the first improvement found. The swap is accepted, the solution is updated, and the entire local search restarts from this new state. This restart begins again with the greedy fill-up phase to ensure the knapsack remains as full as possible. If testing all possible items j yields no improvement for a specific item i, the algorithm reverts to Step 1 and selects the next item $i \in S$. The process terminates only when a full pass over all items in S results in no improving moves, confirming that a local optimum has been reached.

3.2.1 Complexity Analysis

The time complexity of the F&L heuristic is $O(n^2c)$, which is determined by the size of the DP table and the operations performed at each state. The table consists of n stages for the items and c states for the capacity. In every state (i, w), the algorithm must evaluate the potential synergy of the current item with the items already selected in the previous state's set. Since this set can contain up to n items, the synergy summation requires $O(n)$ operations, resulting in a total time complexity of $n \cdot c \cdot n = O(n^2c)$. The space complexity is $O(nc)$ because the algorithm stores the maximum profit and the corresponding item set for every possible weight capacity in every stage (Fomeni and Letchford, 2014).

3.3 Limitations of the F&L Heuristic

While the F&L heuristic demonstrates strong performance on certain instance classes, it suffers from structural limitations that can lead to arbitrarily poor solution quality. We identify two fundamental weaknesses.

3.3.1 Static Ordering

The F&L algorithm computes the item ordering once at initialization using the $\tilde{\pi}_i/w_i$ ratio. This ratio measures each item's total profit potential by assuming all other items might be selected and then dividing by its weight. However, as the algorithm progresses and specific items are chosen or excluded, the potential remaining value of items changes substantially.

Consider an item j with high $\tilde{\pi}_j/w_j$ due to interactions with many other items. The static ordering ranks j highly and processes it early. However, suppose the DP has already excluded most of j's interaction partners (because they had lower $\tilde{\pi}/w$ ratios and were processed later, after capacity was consumed). Item j's actual remaining value is now much lower than $\tilde{\pi}_j/w_j$ suggested. On the other hand, an item k with modest $\tilde{\pi}_k/w_k$ might have strong

interactions with just a few specific items. If those items happen to be selected early, item k becomes highly valuable. But the static ordering, computed before any selections were made, ranked k low based on its total potential across all items.

Hidden clique instances (Schauer, 2016) demonstrate this limitation clearly. These instances contain a planted clique, which is a subset of items with strong mutual interactions. The optimal solution is to select the entire clique. However, clique items individually have low $\tilde{\pi}/w$ ratios because their interactions are concentrated within the clique rather than spread across all items. An item with strong interactions with 20 clique members but zero interaction with 180 non-clique items has lower $\tilde{\pi}$ than an item with modest interactions with all 200 items. The static ordering ranks clique items low. By the time they are processed, the knapsack is already filled with non-clique items, which causes the algorithm to never find the clique.

3.3.2 Greedy Selection in the DP

At each state (i, w), the F&L algorithm maintains only the single solution with the highest current profit for a given weight w using items $\sigma(1), \ldots, \sigma(i)$. By strictly optimizing for immediate profit, the algorithm fundamentally risks discarding promising solutions that hold strong future interaction potential. We will now provide a concrete example to demonstrate how this greedy approach can lead to the selection of a suboptimal set of items.

Suppose at state (i, w) we have:

- Solution S_1: current profit 100, contains item a
- Solution S_2: current profit 95, contains item b

The greedy rule discards S_2 and keeps only S_1. Now suppose item $\sigma(i+1)$ has $q_{i+1,a} = 0$ but $q_{i+1,b} = 25$. If this item is added:

- $S_1 \cup \{\sigma(i+1)\}$: profit $100 + q_{i+1} + \frac{1}{2}(0)$
- $S_2 \cup \{\sigma(i+1)\}$: profit $95 + q_{i+1} + \frac{1}{2}(2 \cdot 25)$

In this scenario, adding item $\sigma(i+1)$ yields an additional profit of 25 for S_2 due to its synergy with item b ($q_{i+1,b}$), while yielding 0 for S_1. This synergy more than bridges the initial gap of 5 units between the two solutions. However, because S_2 was already discarded during the greedy selection at state (i, w), this high-quality solution path is permanently lost. This limitation is fundamental to the one-solution-per-state approximation. No amount of careful static ordering can prevent the greedy rule from occasionally discarding solution paths that would have eventually led to the global optimum.

3.4 Unbounded Approximation Ratio

To evaluate the worst-case performance of the F&L heuristic, we construct a parametric instance family. This allows us to demonstrate that the algorithm's

greedy nature and its fill-up-and-exchange procedure can be systematically trapped in a highly suboptimal solution.

For any parameters M and K such that $M \geq K \geq 3$, we define an instance $\mathcal{I}_{M,K}$ with the knapsack capacity $c = 2M$. The instance consists of three types of items: a single Key item, $K-1$ Bait items, and a single Payload item, resulting in a total instance size of $N = K + 1$. The properties are defined as follows:

Item	Weight	Profit (q_i)	Synergies	$\tilde{\pi}/w$
Key	1	0	M^2 with Payload	M^2
Baits	1 each	$\frac{1}{2}M$ each	none	M
Payload	$2M-1$	0	M^2 with Key	$\approx M/2$

The instance $\mathcal{I}_{M,K}$ is fully characterized by the profit matrix $Q \in \mathbb{Z}^{(K+1)\times(K+1)}$ and weight vector $w \in \mathbb{Z}^{K+1}$, defined as:

$$Q = \begin{pmatrix} 0 & 0 & \cdots & 0 & M^2 \\ 0 & M & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & M & 0 \\ M^2 & 0 & \cdots & 0 & 0 \end{pmatrix}, \qquad \mathbf{w} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ 2M-1 \end{pmatrix}$$

where the diagonal entries $q_{ii} = M$ for $i = 2, \ldots, K$ correspond to the $K-1$ Bait items, and $q_{1,K+1} = q_{K+1,1} = M^2$ encodes the synergy between the Key and Payload.

The core theoretical trap of this instance lies in the static ordering and the capacity constraints. The F&L initialization phase ranks the items based on their $\tilde{\pi}/w$ ratio, resulting in the processing order: Key $\rightarrow$ Baits $\rightarrow$ Payload. Because the Payload has a large weight of $2M-1$, it can only fit into the knapsack if the remaining available capacity is sufficiently large. Theoretically, the fill-up-and-exchange procedure is fundamentally ill-equipped to escape this specific trap. If the dynamic programming phase fills the knapsack with multiple Bait items, adding the Payload would require the simultaneous removal of multiple Baits to free up enough capacity ($2M-1$). Since the local search strictly evaluates 1-exchange moves, it cannot evaluate such coordinated, multi-item swaps, leaving the algorithm permanently stuck in a local optimum. The true optimal solution is to select the Key and the Payload. Together, their combined weight is $1 + (2M-1) = 2M$, which exactly fits the capacity c. Their combined profit is driven entirely by their synergy, yielding an optimal objective value of:

$$\mathrm{OPT}(\mathcal{I}) = M^2$$

Before analyzing the exact execution of the algorithm on this instance, we formally define the concept of an unbounded approximation ratio. For a maximization problem, a heuristic has an unbounded approximation ratio if

its performance can be arbitrarily worse than the optimal solution. Formally, this means that for every integer $u \in \mathbb{N}$, there exists an instance $\mathcal{I}$ such that:

$$\frac{\mathrm{OPT}(\mathcal{I})}{\mathrm{HEUR}(\mathcal{I})} > u$$

In the remainder of this proof, we use $\mathrm{FL}(\mathcal{I})$ to denote the objective value returned by the F&L heuristic on instance $\mathcal{I}$, so that $\mathrm{HEUR}(\mathcal{I}) = \mathrm{FL}(\mathcal{I})$ in this context.

Theorem 1 (Unbounded Ratio of F&L)**.** *The F&L heuristic does not achieve a bounded approximation ratio for the Quadratic Knapsack Problem.*

Proof. We prove this by tracking the exact execution of the F&L heuristic on the instance $\mathcal{I}_{M,K}$ in practice. As established above, the optimal value is M^2 by selecting the Key and the Payload. However, the F&L dynamic programming phase processes the items according to the static ordering, yielding a completely different result:

First, the Key is processed at $i = 1$. Since $q_{\mathrm{Key}} = 0$ and no other items are currently in the solution to provide synergy, the DP state is recorded as $f[1, 1] = 0$ with the corresponding item set $S[1, 1] = \{\mathrm{Key}\}$. Subsequently, the Baits are processed. When the first Bait arrives at $i = 2$, the algorithm evaluates the include option against the current state. The Bait offers a guaranteed immediate profit of $q_{\sigma(2)} = \frac{1}{2}M > 0$, and the greedy selection rule yields:

$$f[2, 1] = \max\{f[1, 1], q_{\sigma(2)}\} = \max\left\{0, \frac{1}{2}M\right\} = \frac{1}{2}M \tag{3.7}$$

This choice effectively replaces the Key in the solution set at capacity $w = 1$, resulting in $S[2, 1] = \{\mathrm{Bait}_1\}$. This logic repeats for the remaining $K - 2$ Baits. Furthermore, this exclusion is not limited to a single state capacity. Across the entire state space, for any evaluated capacity w, an all-Bait subset yields a profit of $\frac{1}{2}wM$, which strictly dominates any subset of equal weight that includes the Key (yielding at most $\frac{1}{2}(w - 1)M$). Because the DP retains only the highest-profit states at every capacity level, the Key is universally discarded from all surviving paths.

By the end of this phase, the DP table reflects a maximum profit of $f[K, K - 1] = \frac{1}{2}(K - 1)M$ using all $K - 1$ Baits. Finally, the Payload is processed at $i = K + 1$. Its weight of $2M - 1$ requires it to combine with a surviving state of weight ≤ 1 to respect the total capacity $c = 2M$. Since the Key has been completely discarded, the only surviving state at this capacity contains a single Bait. Because the Payload lacks the Key, it does not trigger the optimal synergy. Consequently, the state admitting the Payload yields a total profit of at most $\frac{1}{2}M$, which is strictly suboptimal compared to the all-Bait path.

The DP phase terminates with a suboptimal solution containing all $K - 1$ Baits, leaving a remaining capacity of $2M - (K - 1)$. The algorithm then enters the greedy fill-up phase. Since $K \geq 3$, the remaining capacity is strictly less than the Payload's weight of $2M - 1$, meaning the Payload cannot be

added. The Key, weighing only 1, easily fits into the remaining capacity and is successfully added to the knapsack. However, because the Payload is absent, the synergy is not triggered, and the Key contributes 0 to the objective value.

Finally, the 1-exchange procedure is practically ineffective at improving the solution because it cannot accommodate the necessary structural changes. Specifically, swapping a single 'Bait' for the 'Payload' is physically impossible under the capacity constraint, as the 'Payload' requires the simultaneous removal of $K-1$ 'Baits' to fit, a transition that fundamentally violates the limitations of a 1-exchange mechanism.
Thus, the heuristic terminates with $\mathrm{FL}(\mathcal{I}) = \frac{1}{2}(K-1)M$.

To prove unboundedness, we evaluate the approximation ratio for a given $K \geq 3$:

$$\frac{\mathrm{OPT}(\mathcal{I})}{\mathrm{FL}(\mathcal{I})} = \frac{M^2}{\frac{1}{2}(K-1)M} = \frac{2M}{K-1}$$

Let $u \in \mathbb{N}$ be an arbitrary integer. To show that the approximation ratio is not bounded by u, we construct an instance by choosing a parameter M such that $2M > u(K-1)$. For this specific instance, the approximation ratio satisfies:

$$\frac{2M}{K-1} > \frac{u(K-1)}{K-1} = u$$

Since such an instance can be constructed for any $u \in \mathbb{N}$, there exists no constant that upper-bounds the approximation ratio. Therefore, the approximation ratio of the F&L heuristic is unbounded. □

3.5 Addressing F&L Limitations: The Approach by Fennich et al. (2024)

Recent work by Fennich et al. (2024) addresses F&L's limitations through multiple enhancements to the DP framework, achieving substantial improvements on hidden clique instances. The Fennich method introduces three key modifications to the F&L heuristic:

Enhanced Value Function with Forward-Looking Evaluation The standard F&L evaluation for including item $\sigma(i)$ in solution S is:

$$V_{\mathrm{F\&L}}(\sigma(i), S) = q_{\sigma(i)} + \frac{1}{2}\left(2 \sum_{j \in S[i-1, w-w_{\sigma(i)}]} q_{\sigma(i),j}\right) \tag{3.8}$$

While the traditional approach only captures the immediate profit generated by an item and its interactions with the current state, Fennich et al. (2024) expanded this valuation. They enhanced the value function with a forward-looking term, which estimates the potential future interactions with unprocessed items. This effectively prevents the algorithm from making selections based solely on immediate rewards.

Propagation of Promising Solutions A key innovation is backward propagation of improvements from higher-weight states to lower-weight states. After updating a state (i, w), the algorithm identifies the item k with the lowest profit-to-weight ratio in the current solution $S[i, w]$, and computes β as the total profit of the solution excluding item k. It then propagates improvements backward as follows:

1. If $\beta > f[i, w - w_k]$, update state $(i, w - w_k)$
2. Repeat until no improvement or reaching an evaluated state

This backward propagation fills gaps in the state space that might be left unexplored when prioritizing high-weight states early. Crucially, the algorithm still maintains one solution per state. Propagation updates states rather than maintaining multiple solutions simultaneously.

Remove-and-Fill-Up The standard F&L fill-up-and-exchange procedure performs 1-exchange: swap one item in for one item out. Fennich et al. (2024) extend this to a remove-and-fill-up procedure:

1. **Remove phase:** Remove item k from the current solution S, freeing residual capacity λ
2. **Fill phase:** Approximately solve a small sub-problem over all items outside S that fit within λ
3. **Exchange phase:** Accept the swap if the fill-up profit exceeds the contribution of the removed item k

This remove-and-fill-up procedure can escape local optima that require removing one item and adding multiple others, a transformation that 1-exchange alone cannot perform. The procedure performs a single pass over all items in S.

Combined Effect The three components improve the solution in the following ways:

- **Enhanced value function** guides the DP to select items with high future potential, not just immediate profit
- **Propagation** allows the DP to fill gaps in the state space that might be left unexplored when prioritizing high-weight states early
- **Remove-and-fill-up** corrects errors requiring multi-item changes

The distinction between the refinements by Fennich et al. (2024) and the proposed methods in this thesis lies in the specific stage of the heuristic they target. Fennich et al. (2024) enhance the quality of individual decisions through

forward-looking value estimation and refine final outcomes via a remove-and-fill-up local search. However, that underlying framework remains fundamentally greedy, maintaining only a single solution per state. In the following sections of Chapter 4, we introduce beam search and dynamic ordering methods that address the selection mechanism and sequencing logic directly. As detailed in Section 4.1, the beam search provides a structural relaxation of the greedy framework by maintaining multiple candidate paths to prevent premature commitment to suboptimal solutions. Complementing this, the dynamic ordering strategy described in Section 4.2 adaptively reorders items to prioritize realized synergies over static potential. These improvements are motivated by the fact that single-path greedy selection has an unbounded approximation ratio. By expanding the search frontier and adapting the processing order, the algorithm preserves the diversity necessary to recover the global optimum for the instance family $\mathcal{I}_{M,K}$.

3.5.1 Computational Complexity of the Fennich Approach

The methodology introduced by Fennich et al. (2024) increases computational requirements to improve accuracy on difficult instances. The core DP heuristic has a time complexity of $O(n^3c)$ because it evaluates a forward-looking value function at each of the nc states. This evaluation involves a greedy sub-heuristic that runs in $O(n^2)$ time relative to the remaining items. Furthermore, if backward propagation is utilized to fill gaps in the state space, the complexity can rise to $O(n^3c^2)$ as updates may go through the entire capacity range. The local search phase, known as remove-and-fill-up, independently requires $O(n^3)$ time. This phase performs a pass over the solution set to remove items and subsequently execute a greedy fill-up and 1-exchange, making it significantly more demanding than standard local search methods. Despite these substantial increases in computational requirements, the space complexity remains $O(nc)$ as the algorithm still maintains only one solution per state (Fennich et al., 2024).

Chapter 4

Proposed Improvements

Having established that the Fomeni-Letchford (F&L) heuristic suffers from unbounded approximation ratios (Theorem 1), we now propose targeted improvements addressing the two fundamental limitations identified in Section 3.3 which are greedy selection and static ordering.

4.1 Beam Search

Beam search generalizes and relaxes the greedy selection of the F&L heuristic by maintaining up to B solutions per state, where B denotes the beam width, inspired by Ow and Morton (1988) who applied this idea to scheduling problems. In the standard F&L approach (effectively $B = 1$), the Dynamic Programming (DP) table stores only the single best solution found for each state (i, w):

$$f[i, w] := \max\{\text{total value of } S \mid w(S) = w, S \subseteq \{1, \ldots, i\}\}$$

Our proposed improvement expands this state to store a set of candidate solutions:

$$\text{Beam}[i, w] := \{(S_1, p_1), \ldots, (S_k, p_k)\} \quad \text{where } k \leq B$$

where each tuple (S_j, p_j) represents a distinct solution S_j achieving profit p_j at weight w. The DP table is initialized with the base case at stage 0, where $\text{Beam}[0, 0]$ contains an empty set with a profit of 0, and all other weight states $w > 0$ are empty. From this point forward, the algorithm iterates sequentially through the items $\sigma(1)$ to $\sigma(n)$. For each new item $\sigma(i)$ and each possible weight capacity w, the algorithm constructs the new state $\text{Beam}[i, w]$ by pooling candidate solutions from two distinct sources. First, it retrieves up to B solutions from $\text{Beam}[i-1, w]$, representing paths where $\sigma(i)$ is excluded. Second, if the weight constraint allows ($w_{\sigma(i)} \leq w$), it retrieves up to B solutions from $\text{Beam}[i-1, w-w_{\sigma(i)}]$ and expands them by including item $\sigma(i)$, dynamically calculating their new profits based on realized synergies. This process creates a temporary pool of up to $2B$ candidate solutions for the current weight w. Before pruning, duplicate item sets are removed from the pool, retaining

only the highest-profit instance of each unique set. These candidates are sorted by total profit, with ties broken by set cardinality, and strictly reduced to retain only the top B solutions, which become the official entries for Beam$[i, w]$. This formulation ensures that for $B = 1$, the algorithm behaves exactly as the standard F&L heuristic, while for $B > 1$, it preserves suboptimal paths that may later yield superior solutions through synergy effects.

On instance $\mathcal{I}_{M,K}$, a beam search with width $B \geq K$ successfully identifies the optimal solution. This can be demonstrated by tracing the algorithm's execution:

After the algorithm processes the Key and all $K - 1$ Baits, state $(K, 1)$ consists of exactly K candidate solutions:

- $\{\text{Bait}_1\}, \ldots, \{\text{Bait}_{K-1}\}$, each with immediate profit of $\frac{1}{2}M$
- $\{\text{Key}\}$ with profit 0

Case $B < K$: The beam retains only the top-B solutions by profit. Since all Baits have an immediate profit of $\frac{1}{2}M > 0$, the Key (profit 0) is discarded. When the Payload arrives, its weight of $2M - 1$ strictly prevents it from being added to the knapsack alongside the accumulated Baits without violating the total capacity $c = 2M$. Consequently, the algorithm must reject the Payload, and the DP outcome explicitly remains the all-Bait solution, yielding a total suboptimal profit of $\frac{1}{2}(K-1)M$. (Note that while the Key is discarded during this stage, its negligible weight allows it to be collected later during the greedy fill-up phase, though this does not increase the final profit). **Case $B \geq K$:** All K solutions fit in the beam, including $\{\text{Key}\}$. When the Payload arrives, beam search evaluates:

$$\text{Solution: } \{Key, Payload\}, \quad \text{Profit: } q_1 + q_N + \frac{1}{2}(2q_{1,N}) = 0 + 0 + \frac{1}{2}(2M^2) = M^2$$

Since $M^2 > \frac{1}{2}M(K-1)$, this dominates all other combinations and is selected as the final solution. This demonstrates that beam search with appropriate width avoids the unbounded ratio: on the same instance where F&L ($B = 1$) fails arbitrarily, beam search ($B \geq K$) finds the optimum.

4.1.1 Multi-Solution Local Search

To ensure the final results are as accurate as possible, a local search is applied at the conclusion of the DP process. The final beam is assembled by collecting, for each rank tier $i \in \{0, \ldots, B-1\}$, the highest-profit solution at rank i across all capacity levels $w \leq c$, yielding up to B structurally distinct starting points for local search. Once the DP table is fully populated, every candidate solution stored within the final beam undergoes the fill-up-and-exchange (Fomeni and Letchford, 2014) procedure described in Section 3.2. By applying this local search to all solutions in the final beam rather than just the single best result, the algorithm explores multiple local optima. This increases the probability of identifying the global optimum, as the local search can correct sub-optimalities in the various paths preserved by the beam search.

4.1.2 Diverse Beam Search

While beam search improves upon the greedy selection limitation of the Fomeni-Letchford (F&L) heuristic by keeping multiple candidate solutions per state, a fundamental weakness remains. At each state (i, w), a standard beam search retains only the top B solutions ranked strictly by profit. On highly structured instances, this causes the beam to converge to structurally similar solutions very early in the dynamic programming process. All B paths tend to select the same items that offer high immediate profit but low long-term synergy. Because of this, the algorithm permanently excludes the globally optimal structure from the beam, regardless of how large the beam width is.

To solve this convergence problem, we propose a diversity-preserving variant of the beam search. This approach explicitly maintains structural variety among the retained solutions. Instead of ranking all candidates together strictly by profit, the beam at each decision state is partitioned into two equal halves. For odd B, the exploitation group receives $\lceil B/2 \rceil$ slots and the exploration group $\lfloor B/2 \rfloor$. The first half, designated as the exploitation group (E), retains the top solutions based on profit. The second half, designated as the exploration group (D), is filled by selecting solutions that are maximally dissimilar to the exploitation group, which is measured by the Jaccard distance (Leskovec et al., 2020) between their respective item sets. The Jaccard distance, $\delta(d, s)$, is the natural dissimilarity measure for binary selection problems and is defined as:

$$\delta(d, s) = 1 - \frac{|d \cap s|}{|d \cup s|} \tag{4.1}$$

This metric captures the proportion of items that differ between two solutions relative to their combined item pool. A distance of 0 indicates identical solutions, while a distance of 1 indicates completely disjoint item sets.

Let C denote the pool of all valid candidate solutions generated at the current state. The exploitation group is formed by selecting the highest-profit candidates from C. The exploration group is then built iteratively from the remaining, unselected candidates. At each step, the algorithm evaluates the unselected candidates and adds the single candidate that maximizes the minimum Jaccard distance to all solutions already secured in the beam. This objective is defined as:

$$d^* = \arg \max_{d \notin E \cup D} \left(\min_{s \in E \cup D} \delta(d, s) \right) \tag{4.2}$$

After each addition to the exploration group, the distance array is updated incrementally. This ensures that the new exploration group members are diverse not only from the exploitation group, but also from each other. While the Jaccard distance successfully forces the algorithm to explore structurally different paths, ties can still occur. Specifically, when multiple candidate paths are all equally different from our current selections (meaning they share the exact same Jaccard distance), the distance metric alone cannot decide which path is best to choose. To solve this, the algorithm uses a straightforward profit-based

tie-breaker. When several paths are tied in diversity, the algorithm selects the candidate with the highest profit.

While the Diverse Beam Search is designed to overcome structural traps, it does not actually provide a theoretical improvement over the standard beam search on the specific trap instance $\mathcal{I}_{M,K}$. We can see why this happens at the very first step of the search, known as state $(K, 1)$. At this point, the algorithm is only looking at single, individual items: one Key item and $K - 1$ Bait items. Because every item is completely separate from the others, the Jaccard distance between any two items is always exactly 1. This means that based on the diversity rule alone, all items are tied. To break this tie, the algorithm looks at the profit. Since the Bait items have a positive profit and the Key item has a profit of zero, the algorithm will naturally prioritize the Baits over the Key. Because of this profit tie-breaker, the Diverse Beam Search acts exactly like the standard beam search at this initial step. Both algorithms require a beam width of at least $B \geq K$ to make sure the Key item survives. This boundary case shows that the diversity mechanism does not give an advantage at the very beginning when all items are completely distinct. Instead, its true purpose is to handle the complex, overlapping combinations that appear later in the search.

4.1.3 Complexity

The standard beam search variant modifies the dynamic programming framework by relaxing the greedy selection rule to maintain up to B candidate paths. The theoretical time complexity for this approach is $O(Bn^2c)$. For every state in the table, the algorithm processes B candidate solutions, generating inclusion and exclusion versions for each. This requires $2B$ synergy calculations that each take $O(n)$ time. While sorting these candidates to identify the top B results adds a factor of $O(B \log B)$ per state, the linear growth of n remains the dominant factor in practical instances where $n > \log B$.

When extending this framework to the diversity-preserving variant, the algorithm introduces additional computational overhead to prevent premature convergence. The theoretical time complexity for this approach increases to $O(B^2n^2c)$. While generating transitions and evaluating synergies still require $O(Bn)$ time per state, the dominant overhead now stems from populating the exploration group. To select $B/2$ diverse candidates, the algorithm must evaluate the Jaccard distance between item sets. Calculating the intersection and union of two sets of size up to n takes $O(n)$ time. By utilizing an incremental distance array, the algorithm avoids redundant comparisons. When a new solution is added to the exploration group, updating the bottleneck distances for the remaining unselected candidates takes $O(Bn)$ time. Over the required $B/2$ iterations, this results in a diversity selection cost of $O(B^2n)$ per state. Multiplying this per-state cost by the $n \cdot c$ table dimensions yields the overall $O(B^2n^2c)$ time complexity.

For both the standard and diversity-preserving variants, the space complexity increases to $O(Bnc)$. Each cell in the $n \cdot c$ dynamic programming table

must store the item sets and profits for all B solutions in the beam.

4.1.4 Beam Width Selection

The beam width B balances exploration against computational cost. As shown in Section 4.1, the width required to find the optimum depends heavily on both the instance structure and the specific pruning strategy used. On the trap instance $\mathcal{I}_{M,K}$, both the standard and the diversity-preserving beam search require a minimal width of $B \geq K$ to prevent the Key from being discarded.

Generally, instances with many competing high-quality solutions require a larger B to maintain sufficient coverage of the search space, while those with sparse interactions may succeed with a very small B. However, continuously increasing B typically yields diminishing returns in solution quality while strictly increasing the computational burden. Chapter 5 investigates these practical trade-offs through empirical testing to determine the optimal balance for various problem structures.

4.2 Dynamic Ordering

F&L's static ordering computes $\tilde{\pi}_i/w_i$ once at initialization, assuming all items might be selected. However, as the DP progresses, certain items are actually selected in high-value states, revealing which synergies are realized versus merely potential. A static ordering cannot adapt to this information. Consider an item with high synergy to a specific partner item. If that partner is selected early, the synergistic item's value increases dramatically. Static ordering, computed before any selections occur, cannot capture this dynamic.

4.2.1 Algorithm

To select the first item, $\sigma(1)$, we utilize the standard static ordering method previously established. For each item $i \in \{1, \ldots, n\}$, we compute an initial upper bound $\tilde{\pi}_i$ based on the continuous quadratic knapsack relaxation:

$$\tilde{\pi}_i = q_{ii} + \max \left\{ \sum_{j \neq i} q_{ij} x_j : \sum_{j \neq i} w_j x_j \leq c - w_i, \quad 0 \leq x_j \leq 1, \quad j \neq i \right\} \tag{4.3}$$

The first item in our sequence is then determined by the highest profit-to-weight ratio:

$$\sigma(1) = \arg \max_{i \in \{1,\ldots,n\}} \left\{ \frac{\tilde{\pi}_i}{w_i} \right\} \tag{4.4}$$

This initial step ensures that the search begins in the most promising region of the feasible space by accounting for the maximum theoretical synergy available to each item.

Once the anchor is established, we shift to a dynamic ordering for the remaining items $\sigma(2), \ldots, \sigma(n)$. After processing items $\sigma(1)$ to $\sigma(i)$, the next

item $\sigma(i+1)$ is selected by evaluating each remaining candidate j based on the current best solution in the DP table. Specifically, we compute a conditional upper bound:

$$p_j(S, R) = \frac{q_{jj} + 2\sum_{k \in S} q_{jk} + \mathrm{UB}(j, S, R)}{w_j}, \tag{4.5}$$

where:

- S is the highest-profit solution across all states, acting as the anchor for evaluating synergies.
- The term $2\sum_{k \in S} q_{jk}$ accounts for fixed synergies already realized with items selected in the anchor set S.
- $\mathrm{UB}(j, S, R)$ is a relaxed potential synergy bound with the set of remaining items R, defined as:

$$\mathrm{UB}(j, S, R) = \sum_{\substack{\ell \in R \setminus \{j\} \\ w_\ell \leq c - w(S) - w_j}} q_{j\ell} \tag{4.6}$$

This approach prioritizes items based on their total connectivity to the remaining pool. By considering all individually feasible synergies rather than enforcing a collective capacity constraint, the ordering phase maintains a strong directional signal toward dense clusters.

On instance $\mathcal{I}_{M,K}$, this approach successfully navigates the trap set by the Baits. We trace the execution:

Iteration 1: Static Initialization The initial item $\sigma(1)$ is selected via the static density $\tilde{\pi}_i / w_i$:

- **Key** $(j = 1)$**:** $\frac{q_{11} + \sum_{l \in R} q_{1l}}{w_1} = \frac{0 + M^2}{1} = M^2$
- **Payload** $(j = N)$**:** $\frac{q_{NN} + \sum_{l \in R} q_{Nl}}{w_N} = \frac{0 + M^2}{2M - 1} \approx \frac{M}{2}$
- **Baits** $(j \in \{2, \ldots, K\})$**:** $\frac{q_{jj} + 0}{w_j} = \frac{M}{1} = M$

Since $M^2 > M$ for $M > 1$, the Key is processed as $\sigma(1)$. After processing, the DP state $(1, 1)$ contains both the empty set $\emptyset$ and $\{\text{Key}\}$, both with profit 0. This tie is broken by cardinality, so $\{\text{Key}\}$ is retained as the anchor for the dynamic ordering in the next iteration.

Iteration 2: Dynamic Ordering With the Key anchored in the current solution set $S = \{\text{Key}\}$, we evaluate the set of remaining items $R = \{\text{Bait}_1, \ldots, \text{Bait}_{K-1}, \text{Payload}\}$ using the priority function in Equation 4.5. The residual capacity is $c_{\text{rem}} = 2M - 1$.

Payload ($j = N$): The item fits exactly ($w_N = 2M - 1$). The interaction with the Key is now a realized, symmetric synergy:

$$p_N(S, R) = \frac{q_{NN} + 2q_{N,Key} + 0}{w_N} = \frac{2M^2}{2M - 1} \approx M + \epsilon, \quad \text{where } \epsilon = \frac{M}{2M - 1} > 0$$

Baits ($j \in \{2, \ldots, K\}$): Each Bait has no synergy with the Key:

$$p_j(S, R) = \frac{q_{jj} + 2(0) + 0}{w_j} = \frac{M}{1} = M$$

Because $\frac{2M^2}{2M-1} > M$ holds for all $M > 1$, the Payload is prioritized over all Baits and selected as $\sigma(2)$. As a result the DP table processes the Key and Payload in the first two iterations, creating a state $(2, 2M)$ with profit M^2. Because $c = 2M$ is now exhausted, no Baits can be added to this partial solution. Comparing the final profits:

- **Optimal Strategy (Key + Payload):** Profit := M^2
- **Bait Strategy (All Baits):** Profit := $\frac{1}{2}(K - 1)M$

Since $M^2 > \frac{1}{2}(K - 1)M$ by construction, the algorithm successfully recovers the optimal solution where the original F&L heuristic failed.

4.2.2 Complexity

The dynamic ordering strategy introduces an additional computational overhead of $O(n^3)$, which remains independent of the knapsack capacity. At the beginning of each of the n stages, the algorithm must re-sequence all remaining items by calculating their priority relative to the current global anchor set. This evaluation includes a realized synergy calculation and a relaxed potential synergy upper bound, both of which require $O(n)$ time per item. Consequently, the total time for the ordering process across the entire algorithm is $n \cdot n \cdot n = O(n^3)$. When combined with the baseline dynamic programming evaluation, the overall time complexity of the algorithm becomes $O(n^2c + n^3)$. This ensures that items with the highest realized synergy are prioritized without fundamentally altering the underlying state space. The space complexity remains $O(nc)$ as the ordering process does not require additional permanent storage beyond the requirements of the standard dynamic programming table.

4.3 The Combined Approach

The proposed improvements target limitations of the F&L framework. While beam search relaxes the greedy selection constraint by maintaining B candidate paths per state, dynamic ordering adaptively resequences items based on the current best-known solution. These mechanisms are synergistic: dynamic ordering ensures that high-synergy items are prioritized in the processing sequence, while beam search provides the structural depth necessary to preserve these items even when they are temporarily outperformed by non-synergistic alternatives.

4.3.1 Algorithmic Integration

The Combined Approach integrates two distinct selection regimes within the broader beam search framework to balance global positioning with local synergy. At the initial step ($i = 0$), the algorithm selects $\sigma(1)$ based on the static upper bound derived from the continuous relaxation in Equation 4.3. This ensures the search is initialized with an item that possesses the highest potential for global optimality. For all subsequent iterations $i = 1, \ldots, n-1$, the selection process transitions to a dynamic regime. The next item $\sigma(i+1)$ is determined by the priority function in Equation 4.5, which uses the current global leader across all beam states, denoted as the anchor set S, to evaluate synergies. This integration allows the ordering to adapt to the specific clusters being explored by the beam search, effectively guiding the search toward better solutions.

The dynamic ordering phase successfully solves the instance $\mathcal{I}_{M,K}$ by identifying the Key as $\sigma(1)$ and the Payload as $\sigma(2)$ due to their high mutual synergy signal. In the first iteration, the Key is added to the beam. Because the Payload is processed immediately thereafter in the second iteration, the optimal pair $\{Key, Payload\}$ is formed before the capacity can be exhausted by the Baits. Consequently, the optimal solution for instance $\mathcal{I}_{M,K}$ is recovered regardless of the beam width B, as the Key solution does not persist long enough in the beam to be evicted by the $K-1$ Baits. By prioritizing the items in this manner, the instance is solved through the combination of dynamic ordering and beam search, ensuring the critical components of the optimum are merged before they can be displaced by locally competitive but globally suboptimal items.

4.3.2 Complexity Analysis

The Combined Approach integrates the adaptive re-sequencing logic within the multi-path Diverse Beam Search framework to address both sequencing and selection limitations. The total theoretical time complexity for this integration is $O(n^3 + B^2 n^2 c)$. The n^3 component represents a constant computational cost associated with the dynamic ordering, which is independent of the beam width B. Since the next item is selected only once per stage using a single global anchor set, this phase requires the same execution time regardless of the number of solution paths maintained. However, the diversity-preserving mechanism introduces a significant computational overhead. Unlike standard beam search, maintaining a diverse beam requires evaluating and comparing candidate paths against one another to compute their structural differences. This pairwise comparison causes the computational cost of the selection phase to scale quadratically with the beam width. Therefore, while the diverse approach is highly effective at avoiding local optima, its execution time grows rapidly as the beam width increases. The space complexity remains $O(Bnc)$, identical to the standalone beam search variant.

4.4 Summary of Improvements

We have proposed three structural enhancements to the F&L heuristic. Their characteristics and theoretical guarantees are summarized in Table 4.1.

Table 4.1: Comparison of proposed structural improvements to the F&L heuristic.

Method	Target Limitation	Complexity	Finds OPT on $\mathcal{I}_{M,K}$?
F&L Baseline	-	$O(n^2c)$	No
Standard Beam Search	Greedy Selection	$O(Bn^2c)$	Yes (if $B \geq K$)
Diverse Beam Search	Local Optima	$O(B^2n^2c)$	Yes (if $B \geq K$)
Dynamic Ordering	Static Sequencing	$O(n^2c + n^3)$	Yes
Combined Approach	Both	$O(n^3 + B^2n^2c)$	Yes

Each of the proposed methods successfully solves the worst-case family $\mathcal{I}_{M,K}$ that witnesses Theorem 1, provided the beam width B scales with the trap size K. While the Combined Approach carries a higher theoretical complexity, its ability to address both sequencing and selection limitations suggests greater robustness in practice. Chapter 5 evaluates these trade-offs through an empirical performance analysis.

Chapter 5

Experimental Results

5.1 Experimental Setup

All experiments were conducted on a machine equipped with an Intel Core i7-10510U CPU (1.80 GHz base, up to 4.9 GHz turbo) and 16 GB of RAM, running a 64-bit Windows environment. The algorithms were implemented in Python 3.10. Exact solutions were obtained using Gurobi version 13 (single-threaded). The source code and raw data underlying all experimental results are available at: `https://github.com/Thijs101/Thesis_QKP_Final`. It is important to note that we do not directly compare our runtimes in seconds with the original results from Fennich et al. (2024), because their algorithms were written in C++. As a compiled language, C++ processes the heavy, repetitive calculations required by dynamic programming significantly faster than Python. Therefore, we evaluate computational efficiency relatively, by comparing the execution time of our proposed heuristic directly against our own Python implementation of the Fennich baseline. A validation of this reimplementation against the published results of Fennich et al. (2024) is provided in Section 5.5.

For the Gallo (5.2.1) and DKS (5.2.2) instance classes, the evaluation strategy is divided into two approaches based on problem size. For small instances ($n \leq 50$), we use the Gurobi optimizer to find the exact optimal solution, with a strict time limit of 600 seconds per instance. For larger instances ($n > 50$), the problem becomes too complex, making it computationally intractable for Gurobi to find guaranteed optimal solutions. Therefore, we evaluate performance on these larger instances through relative comparison. Finally, for the hidden clique instances (5.2.3), we do not need a solver at all. Because of how these instances are generated, the optimal solution is already known beforehand. For instances with known optimum, we evaluate the heuristics by measuring how close its solutions get to this known theoretical optimum. For each problem instance, let v denote the solution value found by the heuristic, and let OPT represent the optimal solution value. The performance is expressed as the optimality gap:

$$\text{gap} = \frac{\text{OPT} - v}{\text{OPT}} \times 100\%$$

To account for instance variability, each algorithmic configuration is evaluated on 30 independently generated instances. This balances computational feasibility with statistical reliability in the reported averages. For each heuristic and instance configuration, we report the average optimality gap, the optimality rate (defined as the fraction of instances achieving an optimality gap of 0%) where applicable, and computational runtime. The remainder of this chapter is structured as follows. Section 5.2 (Instance Classes) describes the three instance classes used throughout all experiments. Section 5.3 (Beam Width Experiments) investigates the impact of the beam width B on algorithmic performance. This includes a direct comparison between the Standard and Diverse Beam Search, followed by an evaluation across all instance classes to formulate a beam width function $B(N)$. Section 5.4 (Comparisons) benchmarks our final heuristic against the baseline method and other algorithmic variants. Section 5.5 (Fennich Validation) validates that our Python reimplementation of the Fennich et al. (2024) heuristic reproduces the behavior of the original C++ implementation reported in the paper. Section 5.6 (Local Search Improvements) evaluates the impact of applying local search procedures on solution quality and computational runtime. Finally, Section 5.7 (Algorithmic Refinement) investigates the anchor sensitivity problem identified in the beam width experiments and proposes a targeted fix.

5.2 Instance Classes

We evaluate all algorithm variants across three distinct instance classes, each exhibiting different structural properties and complexities:

5.2.1 Gallo Instances

The standard benchmark instances follow the generation scheme of Gallo et al. (1980), as described by Fennich et al. (2024). Instances are generated for varying sizes n, with the synergy density $d \in \{0.25, 0.50, 0.75, 1.0\}$, where d denotes the probability that a synergy q_{ij} exists between any two items i and j. Item weights are drawn uniformly from $[1, 50]$, and the knapsack capacity is drawn uniformly from $[50, \sum_i w_i]$. Both diagonal profits q_{ii} and off-diagonal synergies q_{ij} are drawn uniformly from $[1, 100]$ with probability d, and set to zero otherwise. To maintain a symmetric formulation, each generated synergy is assigned to both q_{ij} and q_{ji}. Under the symmetric objective of Section 3.1, the diagonal entries satisfy $q_{ii} = 2q_i$, so each selected item contributes an effective linear profit of $\frac{1}{2}q_{ii} \in [0.5, 50]$ to the objective.

5.2.2 Densest k-Subgraph (DKS) Instances

The Densest k-Subgraph (DKS) problem asks for a subset of exactly k items that maximizes the total number of synergies within the subset. This problem is NP-hard and maps naturally to the QKP as follows. Each vertex in the graph corresponds to an item with unit weight $w_i = 1$. A synergy $q_{ij} = q_{ji} = 1$

is assigned if a connection exists between items i and j, and $q_{ij} = 0$ otherwise. All diagonal profits are set to zero since there are no individual item profits. The knapsack capacity is set equal to k, so the algorithm must select exactly k items. Random instances are generated with varying sizes n and synergy densities $d \in \{0.50, 0.75, 0.90\}$, where d denotes the probability that a synergy exists between any two items. Following Pisinger (2007), the subgraph size k is drawn uniformly at random from $[2, n-2]$ for each instance.

5.2.3 Hidden Clique Instances

To evaluate performance on instances known to be structurally complex for standard algorithms, we use the hidden clique construction by Schauer (2016). A clique of size $k = \lfloor \sqrt{n} \rfloor$ is planted at a uniformly random position within a graph of n nodes, where each node corresponds to an item with unit weight $w_i = 1$. The knapsack capacity is set to $c = k$, so the known optimal objective value is $\text{OPT} = \frac{1}{2}k(k-1)$, attained by the planted clique and, for small N, possibly by other fully connected k-subgraphs in the random background (see Section 5.3.4). The synergy matrix Q is constructed in two steps. First, a random background is generated by setting each off-diagonal synergy $q_{ij} = q_{ji} = 1$ independently with probability 0.5, and $q_{ij} = 0$ otherwise. Second, all pairs (i, j) within the planted clique are set to $q_{ij} = q_{ji} = 1$, overwriting the random background where necessary. All diagonal profits are set to $q_{ii} = 0$ for every node. Because the clique is hidden within the random background noise, the algorithm has no direct access to the clique's identity and must recover it purely from the structure of Q.

5.3 Beam Width Experiments

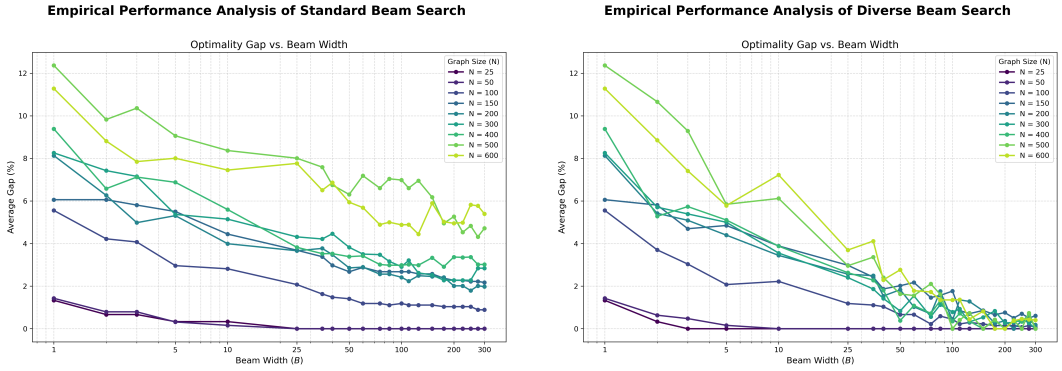
5.3.1 Diverse Beam vs. Standard Beam

Before comparing our final heuristic against the baseline literature, we first investigate how the beam width B affects solution quality, and whether the diversity mechanism is necessary. In Section 4.1, we introduced beam search to relax the greedy selection of the F&L heuristic. The motivation was that maintaining multiple candidate solutions per state would reduce vulnerability to local optima. However, we suspected that a standard beam search would still fail on structured instances like the hidden clique. If candidate selection is based purely on immediate profit, all B solution paths will rapidly converge to structurally similar solutions, making the additional paths redundant. To test this, we compared the Standard Beam Search against our Diverse Beam Search on hidden clique instances across sizes $N = 25$ to $N = 600$, with beam widths ranging from $B = 1$ to $B = 300$. Figure 5.1 illustrates the results of this comparison.

For the Standard Beam Search (Figure 5.1a), increasing the beam width does not reliably improve solution quality on larger instances. For $N \geq 300$, the optimality gap initially decreases as B grows, but quickly plateaus be-

tween 2% and 5% and does not improve further, even at $B = 300$. Because candidate selection is based strictly on profit, all paths concentrate around the same high-profit subgraph, which is typically a dense region of background noise rather than the planted clique. The algorithm therefore fails to maintain meaningful coverage of the search space regardless of beam width. The Diverse Beam Search (Figure 5.1b) resolves this by partitioning the beam into an exploitation group and an exploration group, as described in Section 4.1.2. By explicitly selecting candidates that are structurally dissimilar to the current beam, the algorithm is forced to maintain coverage across different regions of the search space simultaneously. As the beam width increases, the optimality gap decreases consistently and reaches 0.0% once the beam is sufficiently wide, across all tested instance sizes.

These results show that diversity is a necessary condition for reliable performance on structured instances, rather than an optional enhancement. A wider beam without diversity provides no meaningful benefit once the paths have converged. For this reason, all remaining experiments in this thesis use the Diverse Beam Search exclusively.



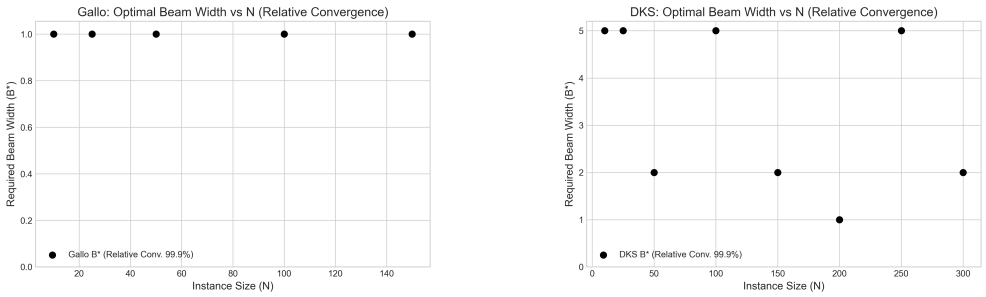
(a) Standard Beam Search (b) Diverse Beam Search

Figure 5.1: Performance comparison across varying beam widths (B). The Standard Beam Search (left) stagnates at optimality gaps between 2% and 5% for larger instances. The Diverse Beam Search (right) forces exploration, successfully driving the gap to 0.0% once a sufficient beam width is used.

5.3.2 Standard Instances

Having established that diversity is required to avoid premature convergence, we now determine the beam width B^* needed for standard benchmark instances. As Schauer (2016) proved, instances generated using the standard methods from Gallo et al. (1980) and standard random instances generally lack deceptive dead ends. This makes them relatively easy to solve even as they grow larger. To confirm this, we tested the algorithm across a range of instance sizes. For small instances, we used the Gurobi Optimizer to find the provably optimal solution (OPT), giving it a strict time limit of 600 seconds per instance. For Gallo instances up to $n \leq 50$ and DKS instances up to

$n \leq 25$, all instances were solved to optimality within the time limit, so no instances were removed. Note that discarding any instance Gurobi cannot prove optimal within the time limit means reported gaps for small instances may be slightly optimistic; this effect is negligible here since all instances were in fact solved. Compared to these exact baseline solutions, our Diverse Beam Search found the perfect solution (a 0.0% gap) using very small beam widths. For larger instances ($n > 50$), exact solvers like Gurobi take too long, so we need a different way to measure success. We define the required beam width (B^*) using a 99.9% relative convergence threshold. This means B^* is the smallest beam width needed to find a solution that is at least 99.9% as good as the absolute best solution we found across all tested beam widths for that specific instance. This approach avoids expending computational effort on marginal improvements, and instead identifies the point at which the algorithm reaches peak performance. The results show that standard benchmark instances are straightforward for the Diverse Beam Search. As shown in Figure 5.2, the required beam width for the Gallo instances stays completely flat at $B^* = 1$. This means greedy choices alone suffice to find the optimal solution. The Densest k-Subgraph (DKS) instances are slightly harder, but never require a beam width larger than $B^* = 5$, regardless of instance size.



(a) Gallo Instances

(b) DKS Instances

Figure 5.2: Required beam width (B^*) to reach peak performance. Gallo instances (left) stay completely flat at $B^* = 1$, while DKS instances (right) vary slightly but always stay below $B^* = 5$.

Figure 5.3 illustrates the convergence rate for DKS instances with density $d = 0.50$, tracking improvement over the $B = 1$ baseline as the beam widens. Performance across all graph sizes reaches within 99.9% of the best solution found at or before $B = 5$. Most importantly, this limit holds true for every density level we tested (Gallo $d \in \{0.25, 0.50, 0.75, 1.0\}$ and DKS $d \in \{0.50, 0.75, 0.90\}$). Even for sparser instances where ties required additional steps to break, the threshold $B^* \leq 5$ was never exceeded. Detailed charts for the remaining densities are provided in Appendix A.

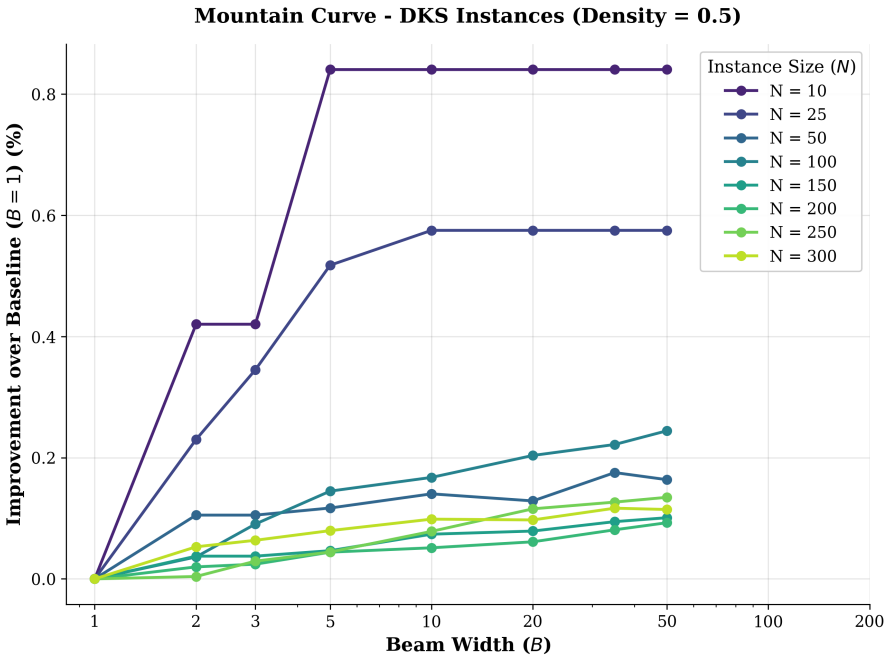


Figure 5.3: Convergence curve for DKS instances ($d = 0.50$). Across all tested graph sizes N, a beam width of $B = 5$ is sufficient to reach within 99.9% of the best solution found.

We therefore use a fixed beam width of $B = 5$ for all standard benchmark instances. As Table 5.1 shows, this width is sufficient to reach near-optimal solutions. Since the beam width does not need to scale with instance size, the additional computational cost is minimal.

Table 5.1: Average performance on standard Gallo and DKS instances ($n \leq 50$) using a fixed beam width of $B = 5$. The baseline represents our combined heuristic restricted to a single solution path ($B = 1$). For DKS instances at $n = 50$, exact solvers ran out of time, so the gaps are compared to the best known solution.

		Greedy Baseline ($B = 1$)		**Diverse Beam ($B = 5$)**	
Class	**Size (N)**	**Gap (%)**	**Time (s)**	**Gap (%)**	**Time (s)**
Gallo	10	0.00	0.002	0.00	0.005
	25	0.06	0.035	0.01	0.240
	50	0.03	0.246	0.01	1.604
DKS	10	0.28	0.002	0.00	0.004
	25	0.42	0.006	0.05	0.025
	50	0.13	0.039	0.05	0.113

5.3.3 Hidden Clique Instances

A fixed beam width of $B = 5$ is sufficient for standard benchmarks, but hidden clique instances behave very differently. The random background noise ($d = 0.50$) in these instances creates many deceptive dead ends that easily trap narrow search algorithms. To solve these difficult instances reliably, we systematically analyzed the data to characterize and model this difficulty profile. To determine the required beam width for hidden clique instances, we ran a Confirmed Crossing study. For every graph size from $N = 10$ to $N = 600$, we identified the minimum beam width B^* needed to consistently achieve an optimality gap below 0.5%. Unlike the standard benchmarks where we use a relative convergence threshold, a higher beam width does not always produce a better solution on hidden clique instances. A wider beam can occasionally find a good solution by chance, making a simple best-found metric unreliable. To avoid this, B^* is defined as the smallest beam width for which the average optimality gap across the three subsequent beam widths remains below 0.5%. This confirms that the algorithm has stably crossed the threshold rather than hit a favorable outlier at a single point. Unlike the standard benchmarks, where a small constant suffices, the hidden clique instances show a striking non-monotone difficulty profile.

As shown in Figure 5.4, the required beam width starts low for small instances, rises sharply, peaks at nearly 250 for $N \in [200, 250]$, and then drops before slowly growing again. To understand why, recall that our instances fix the clique size at $k = \lfloor \sqrt{N} \rfloor$. For small graphs, the clique takes up a large part of the total graph. This makes clique nodes easy to spot, so only a small beam width is needed. As the graph grows, the ratio $k/N \sim N^{-1/2}$ shrinks rapidly. The clique becomes a much smaller fraction of the total graph, and clique nodes start to look more and more like the random background noise. At the same time, the total number of edges inside the clique is not yet large enough to give the algorithm a clear signal. This is where the difficulty peaks. As the graph grows even further, the clique contains enough edges that the Dynamic Programming (DP) heuristic can identify clique nodes reliably again, and the required beam width drops as a result. This tension between clique size and signal strength, studied at the $k \approx \sqrt{N}$ threshold by Alon et al. (1998), produces the observed mountain shape. A formal analytical derivation of this difficulty profile, grounding the mountain shape in the background clique count and a signal-to-noise argument, is provided in Section 5.3.4. Because a simple monotone function cannot capture this shape, we fit a combined Gaussian-linear model. The Gaussian component captures the difficulty peak, while the linear component handles the slow growth at larger scales. Fitting this model to our data yields:

$$B_{\text{base}}(N) = 0.21N + 201 \cdot \exp\left(-\frac{(N - 217)^2}{2 \cdot 74^2}\right) \tag{5.1}$$

with $R^2 = 0.832$. As shown by the solid red line in Figure 5.4, the model accurately tracks the required beam width across the full range of graph sizes.

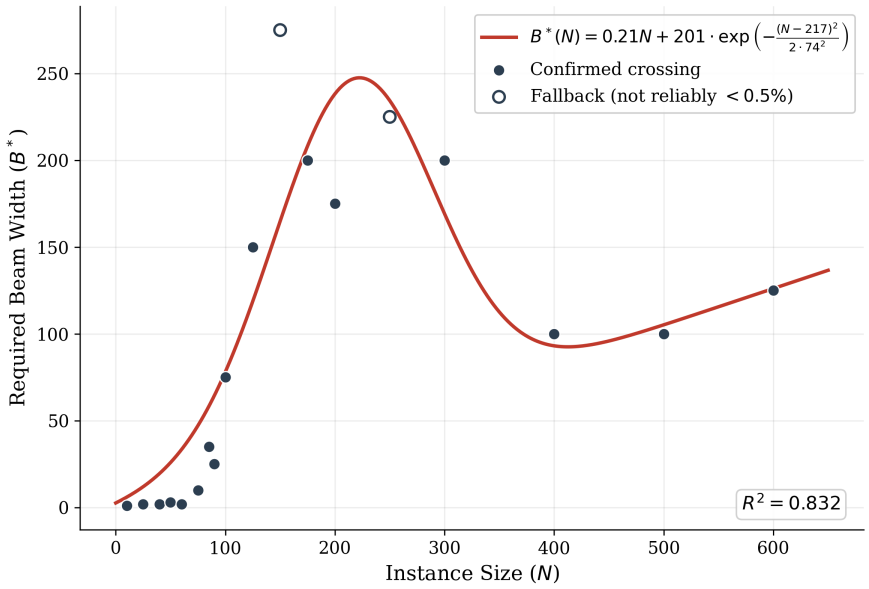


Figure 5.4: Gaussian-linear regression model fitted to the raw confirmed crossing data ($R^2 = 0.832$). The solid red line tracks the average required beam width (B_{base}) across the computational phase transition, capturing the difficulty mountain around $N = 217$.

The Gaussian model correctly predicts the average beam width we need, but using only an average is not enough. Because the background noise in these instances is random, two instances of the exact same size N can have very different difficulty levels. Some instances are considerably more difficult and fall well above the average line. Using only the average beam width risks failure on the more difficult instances. To ensure reliability, we conducted a two-stage Buffer Optimization Study on large instances ($N = 300$ to $N = 600$), testing buffer values from +0% to +10% in steps of 2% to examine how the optimality gap and optimality rate respond. Because the results varied substantially between the 4% and 6% marks, we conducted a focused test at the midpoint (+5%) to identify a stable value. Table 5.2 shows the combined results of these tests.

Table 5.2: Two-stage sensitivity analysis of varying safety buffers for large-scale instances ($N \in [300, 600]$). The study includes a wide search in 2% steps and a focused test at the 5% mark.

	Base (+0%)		**Buffer (+2%)**		**Buffer (+4%)**		**Buffer (+5%)**		**Buffer (+6%)**		**Buffer (+8%)**		**Buffer (+10%)**	
Size (N)	**Gap**	**OPT**	**Gap**	**OPT**	**Gap**	**OPT**	**Gap**	**OPT**	**Gap**	**OPT**	**Gap**	**OPT**	**Gap**	**OPT**
300	0.56%	93.3%	0.25%	96.7%	0.00%	100.0%	0.49%	93.3%	0.27%	93.3%	0.27%	93.3%	0.27%	93.3%
400	0.00%	100.0%	0.68%	93.3%	0.74%	93.3%	0.33%	96.7%	0.39%	96.7%	0.35%	96.7%	0.75%	93.3%
500	1.21%	90.0%	0.00%	100.0%	0.46%	96.7%	0.42%	96.7%	0.38%	96.7%	1.20%	90.0%	1.59%	86.7%
600	1.35%	90.0%	1.33%	90.0%	0.91%	93.3%	0.39%	96.7%	0.92%	93.3%	0.00%	100.0%	1.32%	90.0%

The results in Table 5.2 show a surprising pattern. One would normally expect the algorithm to perform better when given more memory. However, our data shows that adding a small buffer like +2% or +4% can actually make the

results worse. For example, at $N = 400$ the base model achieves a 0.00% gap, yet a 4% buffer raises it to 0.74%. This happens because of a reinforcement effect in the dynamic ordering mechanism. When a distractor path d achieves the highest profit in the beam at stage i, it becomes the anchor for stage $i+1$. The dynamic ordering then prioritizes items with high synergy to d, increasing d's profit relative to all other paths in the beam. This positive feedback loop causes the true optimal path to fall below the beam's profit threshold and be pruned before sufficient instance structure is revealed to identify the planted clique. A larger buffer avoids this by giving the beam enough room to maintain both the distractor paths and the true optimal path simultaneously, preventing any single path from monopolizing the anchor. We will discuss a concrete fix for this reinforcement problem in Section 5.7.

The +5% buffer is where the algorithm becomes stable. Smaller buffers occasionally reach a 0% gap, but vary considerably across instances. The +5% mark consistently keeps the optimality rate at 93.3% or higher and the optimality gap strictly below 0.5% for all tested instance sizes. We therefore use this as the standard setting for all remaining experiments. It should be noted that this calibration is based on a limited number of data points across the tested size range, so the resulting formula should be interpreted as an empirical guideline rather than a theoretically derived scaling law. This gives us the following guideline, which we call the Safe Scaling Rule:

$$B^*(N) = \lceil (1 + \alpha) \times B_{\text{base}}(N) \rceil \tag{5.2}$$

where $\alpha = 0.05$ is the safety buffer established above. Figure 5.5 shows how this buffer raises the prediction line just enough to cover the hardest instances in our data.

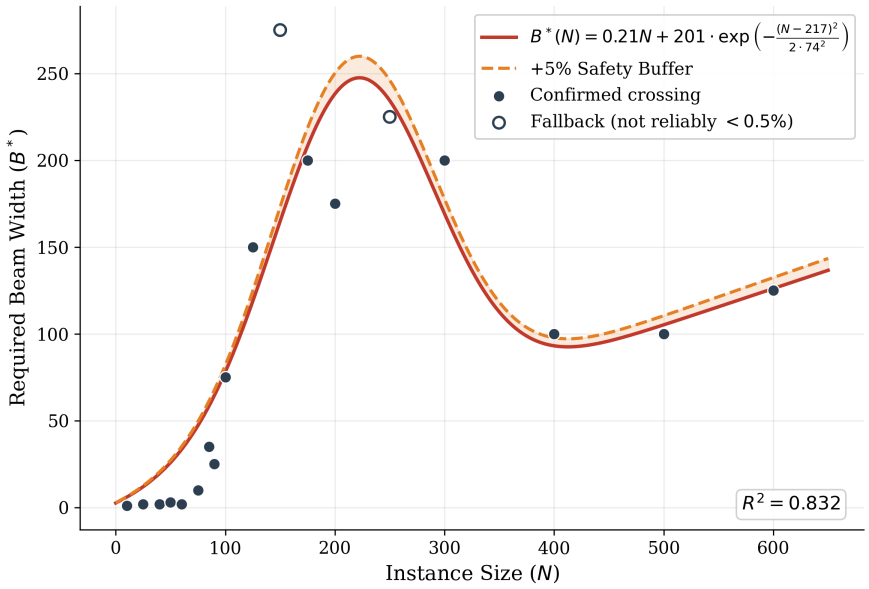


Figure 5.5: The final Safe Scaling Rule including the safety buffer. The dashed line shows how this buffer provides a safety margin, covering the exceptionally hard instances that sit above the average line.

To test whether the scaling formula generalizes beyond its training range, we ran a final validation on hidden clique instances from $N = 700$ to $N = 1200$. These sizes were never used to construct the model, making this a true out-of-sample test. Table 5.3 compares the unbuffered baseline ($\alpha = 0$) against the Safe Scaling Rule ($\alpha = 0.05$).

Table 5.3: Out-of-sample validation on large hidden clique instances ($N \in [700, 1200]$). The optimality rate is the percentage of instances achieving an optimality gap of 0%.

	Unbuffered Baseline ($\alpha = 0$)		**Safe Scaling Rule** ($\alpha = 0.05$)	
Size (N)	**Avg Gap (%)**	**Optimality Rate (%)**	**Avg Gap (%)**	**Optimality Rate (%)**
700	0.98	93.3	1.91	86.7
800	1.02	93.3	1.02	93.3
900	0.57	96.7	0.00	100.0
1000	1.66	90.0	1.69	90.0
1100	0.59	96.7	1.16	93.3
1200	0.59	96.7	1.78	90.0

Both variants continue to perform well beyond the training range, which suggests that the Gaussian-linear model provides a reasonable representation of the difficulty structure rather than just overfitting to the training data. Across all six sizes, both methods maintain optimality rates of at least 86.7% and average gaps below 2%, without needing any retraining. The Safe Scaling Rule shows its clearest advantage at $N = 900$, reaching an optimality rate of 100%. However, as the instances grow larger ($N = 1100$ and $N = 1200$), the unbuffered baseline actually performs slightly better, while the added buffer leads to a somewhat larger average gap. This drop in performance likely happens

because the planted clique signal naturally becomes stronger at these larger sizes. By using a safety buffer and forcing a wider beam, the algorithm might be keeping too many suboptimal paths active, which introduces unnecessary noise. This inconsistency is consistent with the empirical nature of the guideline: the buffer was calibrated for the ambiguous intermediate range and is not expected to provide a uniform benefit across all scales. Overall, these results confirm that the Scaling Rule generalizes well, but they also suggest that the safety buffer is most helpful during the more ambiguous phases of the search and may become less necessary as the instances grow very large.

5.3.4 Analytical Derivation of the Non-Monotone Difficulty Profile

We explain why $B_{\text{base}}(N)$ follows a non-monotone mountain profile using the background clique count $\mathbb{E}[X_k]$, the planted clique advantage $Z(N)$, and a signal-to-noise argument, referring throughout to the hidden clique instance construction of Section 5.2.3, where background edges are present independently with probability 0.5. The analysis relies on expected-value calculations and extreme-value approximations, constituting an analytical model rather than a formal proof, as the DP heuristic remains stochastic and instance-dependent.

The background clique count $\mathbb{E}[X_k] = \binom{N}{k} 2^{-k(k-1)/2}$ measures how many fully connected background k-subgraphs are expected to exist, where $2^{-k(k-1)/2}$ is the probability that all $\binom{k}{2}$ edges within a given k-subgraph are present. When $\mathbb{E}[X_k] > 1$, there is expected to be more than one fully connected background k-subgraph, each attaining the same optimal profit $\frac{1}{2}k(k-1)$ as the planted clique. The planted clique is then not the unique optimal solution, so recovering it specifically offers no advantage. When $\mathbb{E}[X_k] < 1$, the planted clique is the unique optimal solution with high probability.

The planted clique advantage $Z(N)$ quantifies this competition in the regime where $\mathbb{E}[X_k] < 1$, so no background subgraph is fully connected but partially connected ones may still approach the planted clique's profit by chance. Since each of the $\binom{k}{2}$ potential edges in a background k-subgraph is present independently with probability 0.5, the profit follows $\text{Binomial}\left(\binom{k}{2}, 0.5\right)$, with mean $\mu = \frac{1}{2}\binom{k}{2}$ and standard deviation $\sigma = \frac{1}{2}\sqrt{\binom{k}{2}}$, which is well approximated by a Normal distribution for large k by the central limit theorem. By standard extreme-value theory, the expected maximum profit over all $\binom{N}{k}$ background subgraphs is $\mu + \sigma\sqrt{2\ln\binom{N}{k}}$ (Vershynin, 2018). Since the planted clique attains the full profit $2\mu = \binom{k}{2}$, the normalized gap between the planted clique and the strongest expected background competitor is:

$$Z(N) = \frac{2\mu - \left(\mu + \sigma\sqrt{2\ln\binom{N}{k}}\right)}{\sigma} = \sqrt{\frac{k(k-1)}{2}} - \sqrt{2\ln\binom{N}{k}}. \tag{5.3}$$

When $Z < 0$, the expected strongest background subgraph is close enough in profit to the planted clique that the DP's ordering heuristic cannot reliably distinguish between them. When $Z > 0$, the planted clique's profit clearly dominates all background competitors, explaining why recovery becomes possible beyond $N^* = 256$.

Table 5.4 reports exact values. For $N < 100$, $\mathbb{E}[X_k] > 1$ so multiple fully connected background k-subgraphs are expected to exist, each tying the planted clique's profit. At $N = 100$ the discrete jump to $k = 10$ pushes $\mathbb{E}[X_k] < 1$, making the planted clique the unique optimal solution, but Z remains negative, hiding its signal and marking the onset of the hard zone, which persists for $100 \leq N \leq 255$. At $N = 256$ the signal emerges ($Z > 0$) and recovery begins. Note that the negative Z values for $N < 100$ do not indicate signal burial: in this regime $\mathbb{E}[X_k] > 1$ and the Normal approximation underlying Z breaks down, so Z is not meaningful there.

Table 5.4: Exact values of $\mathbb{E}[X_k]$ and $Z(N)$ at key instance sizes.

N	k	$\log_2 \binom{N}{k}$	$\log_2 \mathbb{E}[X_k]$	$Z(N)$	Regime
64	8	32.04	+4.04	−1.37	Multiple optima
100	10	43.98	−1.02	−1.10	Hard onset
150	12	57.26	−8.74	−0.79	Hard
225	15	76.27	−28.73	−0.04	Hard
256	16	83.06	−36.94	+0.22	Recovery
400	20	111.10	−78.90	+1.37	Easy
600	24	141.79	−134.21	+2.59	Easy

The recovery threshold $N^* = 256$ can be derived directly. The largest clique in $G(N, 1/2)$ has size approximately $2\log_2 N$ (Bollobás, 2001, Theorem 11.4), and the planted clique of size $k = \lfloor\sqrt{N}\rfloor$ first exceeds this natural clique number when $\sqrt{N} = 2\log_2 N$. Setting $u = \sqrt{N}$ gives $2^{u/4} = u$, with unique solution $u = 16$ in the relevant range $u > 2$ (since $2^{16/4} = 2^4 = 16$), hence $N^* = 16^2 = 256$. This threshold coincides exactly with the point where $Z(N) = 0$: when $k = \lfloor\sqrt{N}\rfloor$ first exceeds the size of the largest clique that random chance produces in the background, approximately $2\log_2 N$, the planted clique becomes distinguishable and its profit signal emerges above the background noise. The two conditions are therefore not independent. The $Z(N) = 0$ crossing and the $N^* = 256$ threshold are the same mathematical fact expressed differently: one in terms of profit signals, the other in terms of clique sizes.

For $N \geq 256$, $B^*(N)$ continues to grow despite $Z > 0$ because the per-stage ordering becomes the bottleneck rather than the global signal gap. At an early DP stage, when the anchor set S contains few selected nodes, the priority of a candidate vertex reduces to its expected degree. A clique vertex connects to its $k-1$ fellow clique members with certainty and to the remaining $N-k$ vertices with probability 0.5, giving expected priority $(k-1) + 0.5(N-k)$. A background vertex connects to all $N-1$ others at density 0.5, giving $0.5(N-1)$.

The priority surplus of a clique vertex is therefore:

$$\Delta\pi = (k-1) + 0.5(N-k) - 0.5(N-1) = 0.5(k-1) \approx 0.5\sqrt{N}, \qquad (5.4)$$

where the approximation uses $k \approx \sqrt{N}$. Background vertex degrees follow Binomial$(N-1, 0.5)$ with standard deviation $\sigma_\pi = 0.5\sqrt{N-1}$. The signal-to-noise ratio (SNR) measures how many standard deviations the clique vertex's priority surplus stands above the background degree noise:

$$\text{SNR} = \frac{\Delta\pi}{\sigma_\pi} = \frac{0.5(k-1)}{0.5\sqrt{N-1}} \to 1 \quad \text{as } N \to \infty, \qquad (5.5)$$

where convergence is from below, with SNR ≈ 0.90–0.94 for $N \in [100, 600]$ as verified numerically. Since signal and noise grow at the same asymptotic rate, the fraction of background vertices that outrank a given clique vertex is:

$$P(W > 1) = 1 - \Phi(1) \approx 0.16, \qquad W \sim \mathcal{N}(0,1). \qquad (5.6)$$

At each stage roughly $0.16N$ background vertices are therefore expected to outrank the true clique vertex, so the beam must retain at least this many candidates to prevent the clique path from being pruned, giving a per-stage baseline $B_{\min}(N) \approx 0.16N$. The fitted linear slope of $0.21N$ slightly exceeds this baseline, which is consistent with two compounding factors: the clique path must survive all $k \approx \sqrt{N}$ ordering decisions in sequence rather than just one, and the diverse beam reserves part of its width for structural exploration rather than pure exploitation. Together these factors account for the excess of $0.05N$ above the single-stage bound, and the full fitted model (repeating Equation 5.1 for convenience):

$$B_{\text{base}}(N) = 0.21N + 201 \cdot \exp\left(-\frac{(N-217)^2}{2 \cdot 74^2}\right)$$

combines this linear baseline with a Gaussian term centered near $N = 217$ that captures the transient hard zone $N \in [100, 256]$ where the planted clique is unique but its signal is not yet detectable.

5.4 Comparisons

This section presents a systematic ablation study across all three instance classes. We isolate the contribution of each proposed component by evaluating the following algorithmic variants:

- **The F&L Baseline**: A static-ordering heuristic utilizing greedy single-path selection.
- **Dynamic Greedy (DG)**: Extends the baseline by implementing dynamic item ordering while retaining the greedy selection framework.
- **Diverse Beam Search (DBS)**: Introduces a Diverse Beam Search with static ordering to isolate the impact of path diversity.

- **The Combined Approach**: Integrates both dynamic ordering and Diverse Beam Search for comprehensive structural optimization.
- **Fennich Baseline**: Included as an external benchmark representing the recent heuristic (Fennich et al., 2024).

For Gallo and DKS instances, performance is measured against the Gurobi optimal solution for instances where it was obtained, making the optimality gap a true gap to the proven optimum. For larger instances where Gurobi is computationally infeasible, performance is measured relative to the best solution found among all evaluated variants. For hidden clique instances, the optimal value OPT $= \frac{1}{2}k(k-1)$ is known by construction. All beam-based variants use the beam width rules established in Section 5.3: $B = 5$ for Gallo and DKS instances, and the Safe Scaling Rule for hidden clique instances. The Fennich baseline is compared directly up to $N = 75$ for Gallo and $N = 200$ for DKS, beyond which its runtime becomes computationally intractable.

5.4.1 Gallo Instances

Tables 5.5 and 5.6 summarise performance across all Gallo instances. The results show that even the simplest variants perform well on this instance class, which is consistent with the theoretical finding by Schauer (2016) that standard Gallo instances are asymptotically easy.

Table 5.5: Ablation study results for Gallo instances where the Gurobi optimal solution was obtained ($N \leq 50$), broken down by density d and size N. Gap (%) is measured against the proven optimum. Bold values indicate the best result(s) for that row.

		F&L Baseline		**DG**		**DBS**		**Combined**		**Fennich**	
N	d	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
10	0.25	**0.000**	0.002	**0.000**	0.002	**0.000**	0.005	**0.000**	0.006	**0.000**	0.023
	0.50	**0.000**	0.002	**0.000**	0.003	**0.000**	0.004	**0.000**	0.007	**0.000**	0.022
	0.75	**0.000**	0.000	**0.000**	0.003	**0.000**	0.005	**0.000**	0.010	**0.000**	0.036
	1.00	**0.000**	0.002	**0.000**	0.004	**0.000**	0.006	**0.000**	0.006	0.182	0.024
25	0.25	**0.000**	0.018	0.113	0.032	**0.000**	0.191	0.033	0.222	0.065	1.339
	0.50	0.044	0.023	0.107	0.033	**0.000**	0.223	**0.000**	0.288	0.114	1.857
	0.75	**0.000**	0.021	**0.000**	0.030	**0.000**	0.231	**0.000**	0.284	0.199	1.822
	1.00	0.020	0.021	0.025	0.031	**0.000**	0.207	**0.000**	0.274	0.042	1.609
50	0.25	0.084	0.106	0.068	0.167	**0.000**	1.181	0.013	1.544	0.051	25.610
	0.50	0.048	0.143	0.047	0.234	**0.000**	1.511	0.016	1.981	0.072	37.924
	0.75	0.023	0.126	0.023	0.195	**0.000**	1.283	**0.000**	1.663	0.140	33.022
	1.00	**0.000**	0.145	**0.000**	0.238	**0.000**	1.686	**0.000**	1.830	0.078	41.486

The most striking result is that the Fennich baseline performs worse than even the simplest proposed variant on Gallo instances. As shown in Table 5.5, Fennich achieves a 0.182% gap at $N = 10, d = 1.00$ where all proposed variants find the optimum, and consistently produces larger gaps than the F&L

Baseline across small instances while running approximately one to two orders of magnitude slower. This is likely because Fennich's forward-looking value function and backward propagation introduce unnecessary complexity for instances that lack deep structural traps. The F&L Baseline already exploits the instance structure effectively, and adding beam width through DBS achieves perfect performance across all densities and sizes in Table 5.5. Notably, DG occasionally produces a slightly larger gap than the F&L baseline on these easy instances, for example 0.113% against the baseline's 0.000% at $N = 25, d = 0.25$. This is expected: on asymptotically easy instances the static ordering already works well, so re-ranking the items during the search mainly disturbs an order that was already good, sometimes leading to a slightly worse solution. This effect disappears entirely once DBS is added, as the Combined Approach finds the optimum in almost every configuration in Table 5.5 regardless of density.

Table 5.6: Ablation study results for Gallo instances where Gurobi was not used ($N > 50$), broken down by density d and size N. Gap (%) is measured relative to the best solution found among all evaluated variants. The Fennich baseline is included up to $N = 75$ only. Bold values indicate the best result(s) for that row.

		F&L Baseline		**DG**		**DBS**		**Combined**		**Fennich**	
N	d	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
75	0.25	0.088	0.304	0.036	0.547	0.037	3.279	**0.000**	4.342	0.048	157.344
	0.50	0.016	0.315	0.021	0.543	**0.000**	3.467	**0.000**	4.522	0.087	167.348
	0.75	0.003	0.384	**0.002**	0.654	0.003	4.196	**0.000**	5.575	0.024	208.073
	1.00	0.005	0.339	0.007	0.586	0.002	3.577	**0.000**	4.817	0.110	187.418
100	0.25	0.022	0.855	0.015	1.420	0.008	9.342	**0.000**	12.238	—	—
	0.50	0.011	0.781	0.002	1.387	0.001	8.628	**0.000**	11.366	—	—
	0.75	0.003	0.798	0.002	1.377	**0.000**	8.684	**0.000**	11.399	—	—
	1.00	**0.000**	0.918	0.001	1.615	**0.000**	9.552	**0.000**	12.085	—	—
150	0.25	0.020	2.486	0.006	4.301	0.007	28.135	**0.000**	37.363	—	—
	0.50	0.005	2.244	0.004	4.022	0.004	25.634	**0.000**	34.504	—	—
	0.75	0.003	2.418	**0.000**	4.362	0.002	28.089	**0.000**	36.552	—	—
	1.00	0.002	2.405	0.002	4.242	**0.000**	26.406	**0.000**	36.546	—	—

For the larger instances in Table 5.6, where performance is measured relative to the best solution found among all variants, the Combined Approach consistently achieves the lowest gap across all tested sizes and densities. This pattern is also visible in Figure 5.6: Panel A shows the gap curves for DBS and the Combined Approach converging to zero already at small instance sizes, while Panel B shows the runtime gap between Fennich and all proposed variants growing with N, making Fennich increasingly impractical despite offering no accuracy advantage on this instance class.

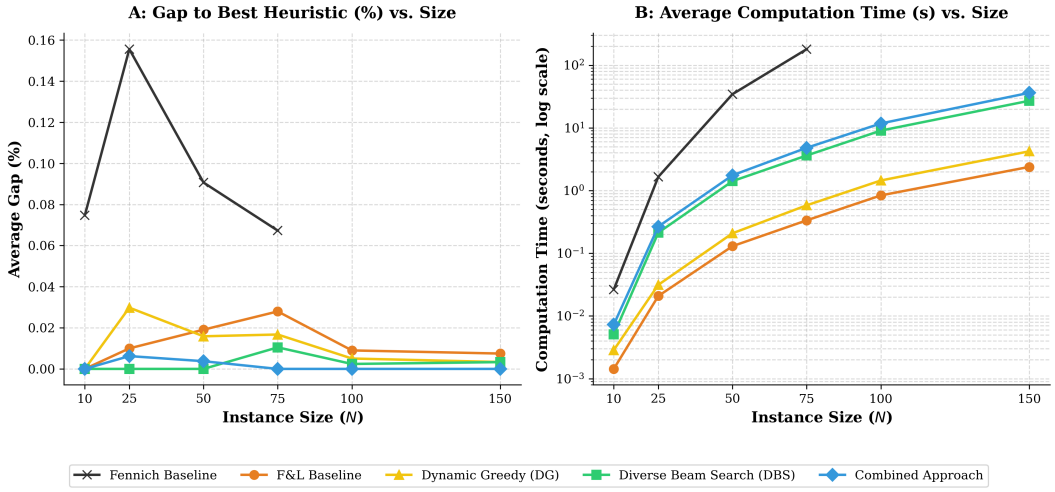


Figure 5.6: Ablation study for Gallo instances. The optimality gap is measured relative to the best-performing heuristic variant across all tested instances. Panel A displays the average optimality gap, while Panel B shows computational runtime on a log scale.

5.4.2 Densest k-Subgraph Instances

Tables 5.7 and 5.8 summarise performance across all DKS instances. This class is more challenging than Gallo due to the binary profit structure and the variable subgraph size k.

Table 5.7: Ablation study results for DKS instances where the Gurobi optimal solution was obtained ($N \leq 25$), broken down by density d and size N. Gap (%) is measured against the proven optimum. Bold values indicate the best result(s) for that row.

		F&L Baseline		**DG**		**DBS**		**Combined**		**Fennich**	
N	d	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
10	0.50	0.859	0.001	0.673	0.001	**0.000**	0.002	**0.000**	0.002	**0.000**	0.004
	0.75	**0.000**	0.000	0.167	0.000	**0.000**	0.002	**0.000**	0.002	**0.000**	0.002
	0.90	**0.000**	0.000	**0.000**	0.001	**0.000**	0.001	**0.000**	0.002	**0.000**	0.004
25	0.50	1.019	0.002	0.934	0.004	0.270	0.015	**0.035**	0.017	**0.035**	0.074
	0.75	0.862	0.002	0.272	0.006	0.234	0.018	**0.100**	0.016	0.127	0.069
	0.90	0.198	0.001	0.043	0.005	0.043	0.011	**0.000**	0.014	0.072	0.053

The contribution of each algorithmic component becomes clearly visible when examining the $N = 25$ instances in Table 5.7, where gaps are measured against the proven Gurobi optimum. (At the smaller $N = 10$ scale, the problem is largely trivialized, with even the F&L Baseline achieving perfect 0.000% gaps at higher densities). At $N = 25$, DG alone consistently reduces the gap compared to the baseline, while DBS drives the most significant independent improvements. The density breakdown reveals how these two components play crucial complementary roles. At low density ($d = 0.50$), where sparse synergies

make the initial item ordering highly consequential, DG proves to be a vital mechanism. While it provides a modest independent improvement over the baseline (reducing the gap from 1.019% to 0.934%), its true impact is revealed when combined with DBS. Relying on DBS alone leaves a gap of 0.270% at this sparsity, but incorporating dynamic ordering (the Combined Approach) slashes this remaining error by nearly an order of magnitude down to 0.035%, equivalent to the gap achieved by Fennich. As density increases, the dense synergy structure makes greedy selection inherently more forgiving. Yet, it is the integration of both mechanisms in the Combined Approach that ultimately masters these denser environments, strictly outperforming Fennich at $N = 25$ for both $d = 0.75$ (0.100% vs. 0.127%) and $d = 0.90$ (achieving a perfect 0.000% vs. 0.072%).

Table 5.8: Ablation study results for DKS instances where Gurobi was not used ($N > 25$), broken down by density d and size N. Gap (%) is measured relative to the best solution found among all evaluated variants. The Fennich baseline is included up to $N = 200$ only. Bold values indicate the best result(s) for that row.

		F&L Baseline		**DG**		**DBS**		**Combined**		**Fennich**	
N	d	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
50	0.50	0.210	0.007	0.117	0.024	0.035	0.082	**0.000**	0.115	0.023	1.525
	0.75	0.430	0.008	0.136	0.027	0.057	0.077	0.023	0.098	**0.000**	1.132
	0.90	0.136	0.006	0.034	0.026	0.009	0.077	**0.000**	0.103	0.009	1.235
100	0.50	0.312	0.037	0.145	0.145	0.018	0.436	**0.000**	0.608	0.009	18.026
	0.75	0.107	0.049	0.086	0.190	0.021	0.546	**0.000**	0.775	0.019	27.723
	0.90	0.082	0.047	0.025	0.201	0.027	0.537	0.011	0.769	**0.000**	25.685
150	0.50	0.172	0.124	0.053	0.593	0.006	1.710	0.006	2.427	**0.000**	169.561
	0.75	0.088	0.105	0.038	0.527	0.006	1.278	0.008	1.860	**0.000**	151.480
	0.90	0.055	0.121	0.014	0.597	0.021	1.453	0.003	2.130	**0.000**	148.116
200	0.50	0.148	0.269	0.044	1.320	0.013	3.650	**0.000**	5.369	0.010	462.960
	0.75	0.105	0.287	0.057	1.298	0.052	3.160	0.024	4.883	**0.000**	470.595
	0.90	0.096	0.262	0.020	1.358	0.030	3.096	0.008	4.862	**0.000**	468.659
250	0.50	0.189	0.428	0.066	2.033	**0.000**	5.249	0.021	7.992	—	—
	0.75	0.114	0.438	0.024	2.279	**0.000**	5.316	**0.002**	9.010	—	—
	0.90	0.049	0.564	0.016	2.838	0.012	6.845	**0.000**	11.941	—	—
300	0.50	0.117	0.861	0.080	4.035	0.023	10.477	**0.000**	16.208	—	—
	0.75	0.062	0.804	0.028	4.417	0.008	9.572	**0.000**	14.837	—	—
	0.90	0.061	0.745	0.018	4.009	0.026	8.841	**0.000**	13.738	—	—

Turning to the larger instances in Table 5.8, where gaps are measured relative to the best solution found among all evaluated variants, the performance trends shift. Fennich achieves the lowest relative gap across all densities at $N = 150$ and for the majority of instances at $N = 200$. However, this accuracy requires significantly higher computational time. At $N = 200$, Fennich's runtime increases to over 460 seconds, whereas the Combined Approach requires approximately 5 seconds while maintaining gaps strictly below 0.025%. Beyond $N = 200$, where Fennich becomes computationally intractable, DBS

and the Combined Approach yield the lowest gaps. DBS alone achieves the best results for the sparser instances at $N = 250$ ($d \in \{0.50, 0.75\}$). As the problem size scales to $N = 300$, the Combined Approach achieves a 0.000% relative gap across all tested densities. Figure 5.7 illustrates these trends: Panel A shows the relative gap decreasing for all variants as N grows, while Panel B displays the runtime crossover, highlighting the difference between Fennich's runtime growth and the proposed variants.

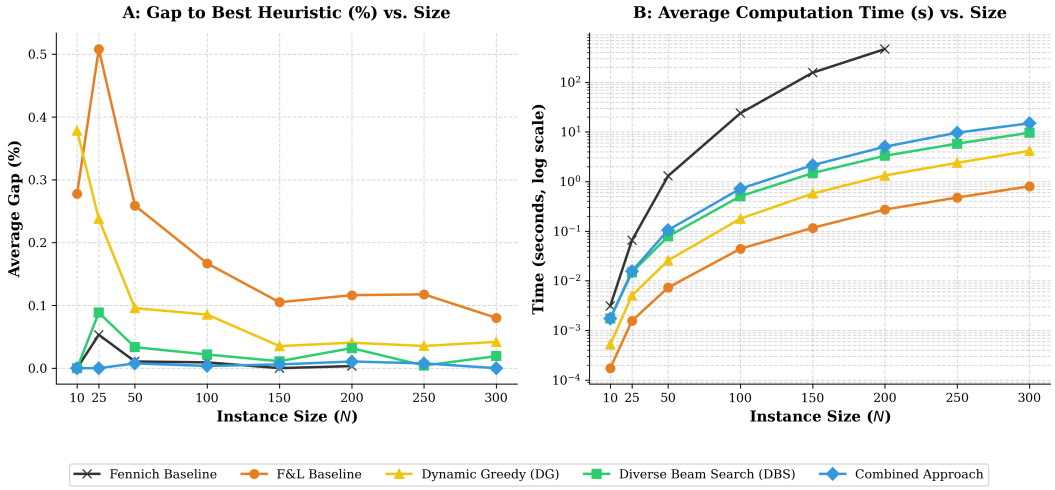


Figure 5.7: Ablation study for DKS instances. The optimality gap is measured against the best-performing heuristic variant identified across all test instances. Panel A displays the average optimality gap across all tested instance sizes, while Panel B shows computational runtime on a log scale.

5.4.3 Hidden Clique Instances

The hidden clique instances represent the primary challenge for all variants. Table 5.9 presents the full breakdown by instance size, while Figure 5.8 shows how performance evolves across instance sizes.

The results clearly illustrate the contribution of each component. The F&L Baseline fails almost entirely on hidden clique instances, achieving at or near 0.0% optimality at $N \geq 90$ and gaps exceeding 16% at $N = 600$, consistent with the structural weaknesses formalized in Theorem 1. DG alone improves performance across all sizes but remains insufficient: at $N = 125$ it drops to 0.0% optimality, and it never recovers meaningfully in the difficulty peak. The largest single improvement comes from adding DBS, which raises the optimality rate to 96.7% at $N = 75$ and maintains above 70% optimality throughout the difficulty peak. This confirms the finding from Section 5.3.1 that diversity is a necessary condition for reliable performance on structured instances.

The Fennich baseline shows a striking U-shaped performance pattern: it struggles severely in the difficulty peak, falling below 50% optimality at $N \in \{100, 125, 150\}$ and reaching as low as 36.7% at $N = 100$, before recovering

strongly beyond $N = 300$ and reaching 100% at $N = 600$. This reflects the fundamental nature of Fennich's single-path forward-looking approach: it either identifies the clique signal or misses it entirely, making it unreliable in the ambiguous middle range where the clique is hard to distinguish from background noise, but highly effective when the signal is unambiguous at larger scales. The Combined Approach does not suffer from this fragility. It maintains robust performance throughout the difficulty peak precisely because the diverse beam preserves multiple structural paths simultaneously, preventing any single distractor from causing a complete failure.

Table 5.9: Ablation study results for hidden clique instances, broken down by instance size N. Gap (%) is measured against the known optimal value OPT $= \frac{1}{2}k(k-1)$. OPT is the optimality rate (%), defined as the fraction of instances achieving an optimality gap of 0%. Time is reported in seconds. Bold values indicate the best result(s) for that row.

	F&L Baseline			**DG**			**DBS**			**Combined**			**Fennich**		
N	Gap	OPT	Time	Gap	OPT	Time	Gap	OPT	Time	Gap	OPT	Time	Gap	OPT	Time
10	**0.000**	**100.0**	0.001	**0.000**	**100.0**	0.000	**0.000**	**100.0**	0.001	**0.000**	**100.0**	0.003	**0.000**	**100.0**	0.001
25	3.000	76.7	0.001	1.333	86.7	0.002	**0.000**	**100.0**	0.016	**0.000**	**100.0**	0.025	**0.000**	**100.0**	0.019
40	3.111	60.0	0.002	1.111	83.3	0.005	**0.000**	**100.0**	0.070	**0.000**	**100.0**	0.114	**0.000**	**100.0**	0.085
50	3.968	36.7	0.003	1.429	76.7	0.010	**0.000**	**100.0**	0.182	**0.000**	**100.0**	0.231	0.159	96.7	0.197
60	3.968	40.0	0.003	1.270	73.3	0.016	**0.000**	**100.0**	0.326	**0.000**	**100.0**	0.412	**0.000**	**100.0**	0.315
75	4.643	16.7	0.007	2.976	46.7	0.024	0.119	96.7	0.846	**0.000**	**100.0**	1.135	0.119	96.7	0.704
85	6.481	13.3	0.008	4.537	16.7	0.039	0.278	90.0	1.627	**0.000**	**100.0**	2.187	1.204	56.7	1.162
90	6.759	0.0	0.006	4.167	20.0	0.043	0.463	83.3	1.969	**0.185**	**93.3**	2.434	1.111	66.7	1.352
100	7.333	0.0	0.010	5.556	10.0	0.053	0.519	76.7	3.485	**0.148**	**93.3**	4.653	1.926	36.7	2.268
125	8.667	3.3	0.024	6.606	0.0	0.098	0.848	63.3	10.378	**0.727**	**66.7**	13.216	2.242	36.7	4.420
150	9.444	0.0	0.018	6.061	20.0	0.130	1.111	70.0	25.552	**0.404**	**83.3**	31.218	2.677	36.7	8.702
175	9.744	3.3	0.029	6.923	16.7	0.191	0.769	83.3	50.026	**0.641**	**83.3**	55.934	1.453	63.3	14.619
200	11.465	3.3	0.033	8.132	10.0	0.267	0.403	**93.3**	82.129	**0.366**	**93.3**	93.614	1.978	50.0	22.976
250	10.889	0.0	0.051	9.714	0.0	0.439	1.524	70.0	112.037	**0.476**	**86.7**	130.479	2.286	50.0	47.657
300	12.917	0.0	0.073	8.260	23.3	0.725	0.784	90.0	85.047	**0.490**	**93.3**	85.829	0.539	86.7	93.314
400	15.246	0.0	0.132	9.386	30.0	1.497	3.526	66.7	51.270	**0.333**	**96.7**	50.873	0.439	90.0	276.177
500	15.498	3.3	0.196	12.367	13.3	2.496	2.424	80.0	87.500	**0.418**	**96.7**	88.570	0.649	80.0	588.108
600	16.280	3.3	0.436	11.280	30.0	6.158	3.563	73.3	161.313	0.386	96.7	163.097	**0.000**	**100.0**	1052.124

The Combined Approach provides a consistent additional gain over DBS, concentrated precisely in the structurally ambiguous difficulty peak. At $N = 75$ and $N = 85$, the Combined Approach achieves a perfect 100% optimality rate while DBS reaches only 96.7% and 90.0% respectively. At $N = 250$, the Combined Approach achieves 86.7% optimality compared to DBS's 70.0%, and at $N = 300$ it reaches 93.3% while DBS achieves 90.0%. These gains demonstrate that dynamic ordering helps identify the clique structure early in the search, before the beam is dominated by distractor paths. Outside the difficulty peak, at $N \geq 400$, the Combined Approach performs strongly ($\geq 96.7\%$), while DBS remains variable (66.7%–80.0%), confirming that the clique signal becomes strong enough for both variants to find it reliably. Panel A of Figure 5.8 illustrates this pattern clearly: the difficulty mountain peaks around $N \in [150, 250]$, where the F&L Baseline, DG, and the Fennich baseline all collapse, while DBS and the Combined Approach maintain robust performance throughout. The advantage of the Combined Approach over DBS is most visible precisely in this peak region.

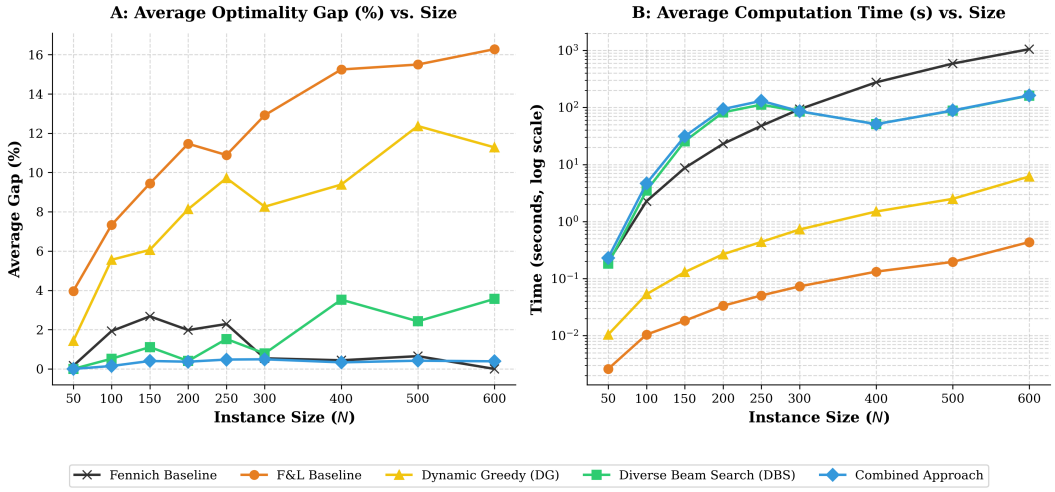


Figure 5.8: Ablation study for hidden clique instances. Panel A shows the average optimality gap across instance sizes. Panel B shows the average computation time on a log scale. The Combined Approach consistently achieves the lowest gap. Runtime remains higher than Fennich through the difficulty peak due to the wide beam required by the Safe Scaling Rule, but diverges favorably beyond $N = 300$ as Fennich's cost grows sharply.

Beyond $N = 300$, the planted clique signal becomes mathematically stronger, allowing all competitive variants to recover. Notably, Fennich achieves 100% optimality at $N = 600$ compared to the Combined Approach's 96.7%. At these extreme scales, the clique signal is so unambiguous that Fennich's forward-looking value function can reliably identify it from a single path without requiring broad beam exploration. However, this single-path accuracy comes at a steep computational cost: at $N = 600$, Fennich requires 1052 seconds compared to the Combined Approach's 163 seconds, and at $N = 500$, the difference is 588 versus 89 seconds. Panel B of Figure 5.8 makes this divergence visible: the runtime curves for DBS and the Combined Approach remain close throughout, while Fennich's curve separates sharply beyond $N = 200$ and grows at a rate consistent with its $O(n^3c^2)$ complexity. This suggests the two approaches are complementary: the Combined Approach is highly efficient and superior in the structurally deceptive middle ranges, while Fennich's guided approach excels when the signal is unambiguous, provided computation time is not a constraint.

5.5 Validation of the Fennich baseline

Since all comparisons in this thesis are conducted against a Python reimplementation of Fennich et al. (2024) rather than the original C++ code, we validate its consistency with the original method here. As Fennich et al. (2024) do not provide their exact instances, a direct replication is not possible. Instead, we compare our reimplementation's average optimality gaps on independently generated hidden clique instances of matching sizes against the published re-

sults of the full method (Novel DP with propagation and remove-and-fill-up local search), which corresponds to the best-performing variant in their paper. Published results are reported over 10 instances per size and our results are averaged over 30 independently generated instances using the same generation parameters: a clique of size $k = \lfloor \sqrt{N} \rfloor$ planted within a random graph of N nodes with background density 0.5.

Table 5.10: Validation of our Fennich et al. (2024) reimplementation on hidden clique instances. Gap (%) is the average optimality gap against the known optimum $\frac{1}{2}k(k-1)$. Published results are from the full method (Novel DP + propagation + remove-and-fill-up) of Fennich et al. (2024), implemented in C++. Our reimplementation is in Python, therefore absolute runtimes are not directly comparable, but the runtime ratio is reported as additional evidence of correct algorithmic scaling.

	Published (Fennich et al., 2024) (C++)		**Our Reimplementation (Python)**		
N	Gap (%)	Time (s)	Gap (%)	Time (s)	Runtime Ratio
50	0.00	0.01	0.159	0.197	20×
100	0.67	0.08	1.926	2.268	28×
150	1.97	0.28	2.677	8.702	31×
200	1.32	0.72	1.978	22.976	32×
250	0.57	1.39	2.286	47.657	34×
300	0.15	2.69	0.539	93.314	35×
400	0.00	7.59	0.439	276.177	36×
500	0.00	17.05	0.649	588.108	34×
600	0.00	33.52	0.000	1052.124	31×

The results confirm that our reimplementation reproduces the behavior of the published method. Both exhibit the characteristic difficulty peak in the $N \in [100, 250]$ range, with gaps rising to approximately 2% before recovering at larger sizes, and both achieve a 0.000% average gap at $N = 600$. The runtime ratio between our Python implementation and the published C++ code remains consistently in the 20–36× range across all tested sizes, which is consistent with the expected overhead of interpreted versus compiled execution and indicates that the reimplementation correctly replicates the algorithmic scaling of the original.

Our reimplementation does produce higher average gaps than the published results at several sizes, most notably in the $N \in [250, 500]$ range where the published method approaches zero while ours remains between 0.4% and 0.7%. We attribute this to two factors: implementation details not fully specified in the paper description alone, and the use of 30 instances per size rather than the original 10, which produces a more representative average and captures a wider range of instance difficulty. Since all comparisons in this thesis evaluate the same reimplementation consistently across all algorithmic variants on identical instances, the relative performance conclusions between methods remain valid.

5.6 Local Search Improvements

To isolate the contribution of the local search phase, we measure the average improvement in solution quality achieved by applying the respective local search procedures to the raw algorithmic output. Specifically, we evaluate the gain from our fill-up-and-exchange procedure applied to the beam search output, and contrast it with the remove-and-fill-up local search employed by the Fennich baseline. To ensure a fair comparison, the averages for both methods are restricted to the same instance size ranges: up to $N = 75$ for Gallo, up to $N = 200$ for DKS, and all sizes for hidden clique instances where Fennich was run throughout. Table 5.11 summarises this average percentage gain across all instance classes for both methods.

Table 5.11: Average local search gain (%) by instance class, measured as the percentage improvement from the raw DP output to the final solution after local search. Averages are restricted to the same size ranges for both methods: up to $N = 75$ for Gallo, up to $N = 200$ for DKS, and all sizes for hidden clique instances.

Instance Class	**Avg LS Gain (%)**		**Avg Total Time (s)**	
	Combined	**Fennich**	**Combined**	**Fennich**
Gallo	0.006	0.005	1.711	54.060
DKS	0.320	0.023	1.336	108.161
Hidden Clique	3.287	0.189	40.223	117.455

The local search gain is negligible for Gallo instances, where the beam search already finds near-optimal solutions directly. For DKS instances the gain is moderate at 0.320% on average. The most substantial gains occur on hidden clique instances, where local search improves our method by 3.287% on average compared to only 0.189% for Fennich. This difference reflects the structural diversity of the beam search output: our method maintains structurally varied solutions across the beam, giving local search a richer set of starting points to improve from. Fennich's single-path output leaves less room for local refinement.

As shown in Figure 5.9, the gain from local search grows substantially with instance size for our method. At $N = 400$ the average gain reaches 8.17% and at $N = 500$ it peaks at 10.43%, before dropping slightly to 9.77% at $N = 600$. The Fennich baseline remains near zero throughout, never exceeding 0.72% at any size. This growing gap reflects a fundamental difference in how the two approaches interact with local search. Our Diverse Beam Search produces solutions that are structurally close to the planted clique but not yet complete. The fill-up phase of local search can then add the remaining clique members within the available capacity, yielding large gains. Fennich's forward-looking single path tends to either find the clique or miss it entirely, leaving little room for incremental improvement through local search.

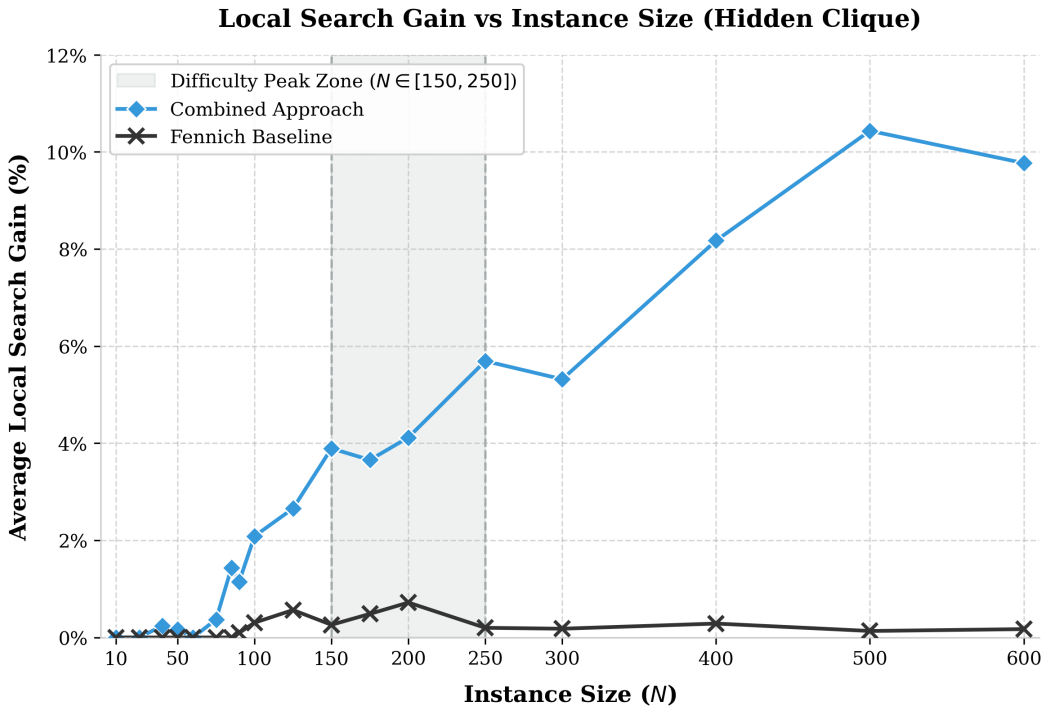


Figure 5.9: Average local search gain (%) as a function of instance size for hidden clique instances. The gain is measured as the percentage improvement from the raw DP output to the final solution after applying the fill-up-and-exchange procedure. The shaded region marks the difficulty peak zone ($N \in [150, 250]$).

Interestingly, the gain does not peak inside the difficulty zone $N \in [150, 250]$ but continues growing well beyond it. This suggests that at larger scales, the beam search consistently finds partial clique solutions that local search can complete, whereas inside the difficulty zone the beam sometimes misses the clique structure entirely, making local search unable to recover it.

5.7 Algorithmic Refinement

Section 5.3.3 identified a reinforcement effect in the dynamic ordering mechanism: when a distractor path achieves the highest profit in the beam, it takes control of the anchor and pulls the ordering away from the true clique before the algorithm has seen the full instance. This section proposes and evaluates a targeted fix, namely the Aggregated Anchor Approach.

5.7.1 Aggregated Anchor Approach

Instead of using a single profit-maximizing solution as the anchor for dynamic ordering, the Aggregated Anchor uses the top p fraction of beam states by profit as a collective anchor. The priority of each candidate item is computed as the average priority across all anchor states:

$$\bar{\pi}(j) = \frac{1}{|\mathcal{A}|} \sum_{S \in \mathcal{A}} p_j(S, R) \tag{5.7}$$

where $\mathcal{A}$ is the set of anchor states comprising the top $\lceil p \cdot B \rceil$ solutions by profit, and $p_j(S, R)$ is the priority of item j given anchor state S and the set of remaining items R, as defined in Equation 4.5. By averaging across multiple anchor states, no single distractor path can fully control the ordering signal. A distractor that dominates one anchor state will be dampened by the remaining states that represent structurally different solutions.

5.7.2 Sensitivity Analysis

To select the fraction p, we ran a sensitivity analysis at $N \in \{150, 200, 250\}$, corresponding to the difficulty peak where differences between variants are most pronounced. We tested five values $p \in \{0.05, 0.10, 0.15, 0.20, 0.25\}$.

Table 5.12: Sensitivity analysis of the aggregation fraction p at the three hardest instance sizes. The optimality rate is the fraction of instances achieving an optimality gap of 0%. Bold values indicate the best result(s) for that row.

	$N = 150$		$N = 200$		$N = 250$	
Fraction p	**Gap (%)**	**OPT (%)**	**Gap (%)**	**OPT (%)**	**Gap (%)**	**OPT (%)**
$p = 0.05$	0.758	76.7	0.183	96.7	0.476	90.0
$p = 0.10$	0.505	80.0	0.183	96.7	0.667	90.0
$p = 0.15$	**0.455**	**83.3**	0.147	**96.7**	**0.222**	**93.3**
$p = 0.20$	0.556	80.0	0.183	96.7	1.270	76.7
$p = 0.25$	0.455	80.0	**0.037**	**96.7**	0.508	86.7

As shown in Table 5.12, $p = 0.15$ performs best or tied for best across all three sizes. Smaller fractions like $p = 0.05$ provide too little dampening, leaving the anchor vulnerable to distractor dominance. Larger fractions like $p = 0.20$ and $p = 0.25$ begin to dilute the ordering signal too much, particularly at $N = 250$ where $p = 0.20$ collapses to 76.7% optimality. The $p = 0.15$ fraction sits at the stable middle ground, providing an effective balance between dampening distractor paths and maintaining heuristic focus. We therefore use $p = 0.15$ for all subsequent experiments.

5.7.3 Full Evaluation

Using $p = 0.15$, we evaluated the Aggregated Anchor Approach across all hidden clique instance sizes $N \in [10, 1200]$ using the same Safe Scaling Rule beam widths established in Section 5.3.3. Table 5.13 compares the Aggregated Anchor variant against the buffered baseline across all tested sizes.

The results show a mixed but informative picture for $N \leq 600$, where both variants are compared under the same Safe Scaling Rule beam widths. The Aggregated Anchor improves performance at $N \in \{90, 125, 175, 200, 250, 300\}$, with the largest gains at $N = 175$ where optimality rate jumps from 83.3% to 96.7%, and at $N = 300$ where it achieves a perfect 100%. These improvements

are concentrated in the peak zone where the reinforcement effect is most damaging, confirming that the Aggregated Anchor successfully dampens distractor dominance in this range. However, it also introduces regressions at $N = 85$ and $N = 100$, where the optimality rate drops from 100% to 86.7% and from 93.3% to 73.3% respectively, suggesting that at these smaller sizes the beam is already sufficiently focused on the true clique and averaging across anchor states introduces unnecessary noise.

Table 5.13: Performance comparison between the buffered baseline (Combined Approach + Safe Scaling Rule, $\alpha = 0.05$) and the Aggregated Anchor Approach ($p = 0.15$) for hidden clique instances with $N \geq 85$. Bold values indicate the best result(s) for that row.

	Buffered Baseline ($\alpha = 0.05$)			**Aggregated Anchor** ($p = 0.15$)		
N	**Gap (%)**	**OPT (%)**	**Time (s)**	**Gap (%)**	**OPT (%)**	**Time (s)**
85	**0.000**	**100.0**	2.19	0.370	86.7	2.20
90	0.185	93.3	2.43	**0.093**	**96.7**	2.79
100	**0.148**	**93.3**	4.65	0.593	73.3	4.85
125	0.727	66.7	13.22	**0.667**	**70.0**	14.38
150	**0.404**	**83.3**	31.22	0.455	**83.3**	28.60
175	0.641	83.3	55.93	**0.085**	**96.7**	68.76
200	0.366	93.3	93.61	**0.147**	**96.7**	94.03
250	0.476	86.7	130.48	**0.222**	**93.3**	129.18
300	0.490	93.3	85.83	**0.000**	**100.0**	125.85
400	**0.333**	**96.7**	50.87	0.737	93.3	85.65
500	**0.419**	**96.7**	88.57	2.035	83.3	151.70
600	**0.387**	**96.7**	163.10	0.882	93.3	286.83
700	1.908	86.7	440.48	**0.000**	**100.0**	425.38
800	1.023	**93.3**	472.51	**1.014**	**93.3**	678.81
900	**0.000**	**100.0**	751.42	**0.000**	**100.0**	1110.64
1000	1.692	90.0	1164.86	**0.545**	**96.7**	1481.96
1100	1.155	93.3	1523.77	**0.000**	**100.0**	2141.42
1200	1.783	90.0	2017.29	**0.000**	**100.0**	2900.13

Furthermore, at larger sizes $N \in \{400, 500, 600\}$ under the constrained beam widths of the Safe Scaling Rule, the Aggregated Anchor performs worse than the buffered baseline. At $N = 500$, the average gap increases from 0.419% to 2.035%. This happens because the Safe Scaling Rule allocates a narrower beam at these scales (around 100), meaning the Aggregated Anchor draws from a much smaller pool of anchor states. At these scales, the single highest-profit state is also likely already tracking the true clique, as the planted clique signal becomes stronger. Averaging this signal across lower-ranked states therefore dilutes the ordering focus and introduces noise.

The extended experiments at $N \in [700, 1200]$ confirm the hypothesis underlying this degradation and paint a more complete picture of the two variants. The Aggregated Anchor achieves a perfect 100% optimality rate at $N \in \{700, 900, 1100, 1200\}$ and 96.7% at $N = 1000$, outperforming the buffered baseline at every size in this range except $N = 800$ and $N = 900$, where both perform comparably. The buffered baseline, in turn, shows its own variabil-

ity at these scales, reaching only 86.7% at $N = 700$ and dropping to 90.0% at both $N = 1000$ and $N = 1200$. These results validate that the Aggregated Anchor strategy is sound in general, and its apparent degradation at $N \in \{400, 500, 600\}$ is an artifact of the beam width constraint imposed by the current Safe Scaling Rule rather than a fundamental weakness of the approach. When provided with a sufficiently wide beam, the Aggregated Anchor consistently matches or exceeds the buffered baseline. Which variant to use therefore depends on the instance size: the Aggregated Anchor throughout most of the difficulty peak and at all sizes from $N = 700$ onward, and the buffered baseline in the intermediate range $N \in [400, 600]$. A scaling rule fitted specifically for the Aggregated Anchor would most likely close this gap; we return to this in Chapter 6.

Chapter 6

Conclusion

This thesis investigated targeted enhancements to the Fomeni-Letchford (F&L) dynamic programming heuristic to restore its performance on Quadratic Knapsack Problem (QKP) instances with complex interactions. We formally proved that the F&L baseline has an unbounded approximation ratio, and introduced a Combined Approach integrating Diverse Beam Search with dynamic item ordering to address its structural weaknesses. To manage the computational cost of this expanded search space, we modeled the non-monotone difficulty profile of hidden clique instances and introduced a Safe Scaling Rule to assign appropriate beam widths. Additionally, an Aggregated Anchor Approach was developed to address reinforcement effects during dynamic ordering.

The Combined Approach is competitive with or better than the Fennich et al. (2024) baseline in solution quality across all tested instance classes, with its clearest advantage on the hidden clique instances, where it maintains robust performance through the structurally ambiguous difficulty peak that causes the Fennich baseline to collapse, while becoming substantially faster at the largest instance sizes. On the standard benchmark classes, differences in solution quality are small, but the computational advantage is substantial across all sizes.

A limitation of this study is that the Safe Scaling Rule and the Aggregated Anchor fraction were calibrated independently, causing the Aggregated Anchor to underperform in the intermediate $N \in [400, 600]$ range where the beam is too narrow for the averaging to help. Additionally, all experiments were conducted in Python, which is substantially slower than the C++ implementation used by Fennich et al. (2024), and the local search is restricted to 1-exchange moves, whereas the remove-and-fill-up procedure of Fennich et al. (2024) supports multi-item swaps.

A natural extension is an adaptive beam width strategy that monitors structural diversity during execution and widens the beam when paths converge prematurely, and narrows it once the optimal signal is clear. This is directly motivated by our signal-to-noise analysis: the required beam width tracks how distinguishable the planted clique signal is from background noise at each instance size, suggesting that an algorithm aware of this signal in real time could self-regulate without any pre-fitted Scaling Rule.

Furthermore, our experiments demonstrated that the Diverse Beam Search is highly complementary to local search, with our simple fill-up-and-exchange procedure already achieving up to approximately 10% improvement in final solution quality on the largest hidden clique instances by completing partial clique structures. Applying the remove-and-fill-up procedure of Fennich et al. (2024) to all structurally diverse solutions in the final beam, rather than a single path, could leverage this diversity to achieve even greater gains.

A final direction for future work concerns the calibration itself. The Safe Scaling Rule and the aggregation fraction were both fitted empirically, but the signal-to-noise analysis in Section 5.3.4 suggests they do not have to be. That analysis already accounts for the linear growth of the required beam width and for the position of the difficulty peak, so it provides a basis for deriving formal convergence guarantees. With such guarantees, the beam width would no longer be an empirical setting but a quantity that follows directly from the structure of the instance.

Technology statement

During the preparation of this work, I used Claude (Anthropic) and Gemini (Google) in order to assist with writing, editing, and polishing the thesis text and Python code. The following parts of the assignment were affected by AI tool usage: Introduction, Preliminaries, Quadratic Knapsack Problem, Proposed Improvements, Experimental Results and Conclusion. After using these tools, Thijs Niekerk evaluated the validity of the tools' outputs, including the sources that generative AI tools have used, and edited the content as needed. As a consequence, Thijs Niekerk takes full responsibility for the content of their work.

Bibliography

Alon, N., Krivelevich, M., and Sudakov, B. (1998). Finding a large hidden clique in a random graph. *Random Structures & Algorithms*, 13(3-4):457–466.

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.

Bellman, R. (1966). Dynamic programming. *Science*, 153(3731):34–37.

Billionnet, A. and Soutif, É. (2004a). An exact method based on Lagrangian decomposition for the 0–1 quadratic knapsack problem. *European Journal of Operational Research*, 157:565–575.

Billionnet, A. and Soutif, É. (2004b). Using a mixed integer programming tool for solving the 0–1 quadratic knapsack problem. *INFORMS Journal on Computing*, 16(2):188–197.

Bollobás, B. (2001). *Random Graphs*. Cambridge University Press, 2nd edition.

Cacchiani, V., Iori, M., Locatelli, A., and Martello, S. (2022a). Knapsack problems — an overview of recent advances. Part I: Single knapsack problems. *Computers & Operations Research*, 143:105692.

Cacchiani, V., Iori, M., Locatelli, A., and Martello, S. (2022b). Knapsack problems—an overview of recent advances. Part II: Multiple, multidimensional, and quadratic knapsack problems. *Computers & Operations Research*, 143:105693.

Caprara, A., Pisinger, D., and Toth, P. (1999). Exact solution of the quadratic knapsack problem. *INFORMS Journal on Computing*, 11:125–137.

Chen, Y. and Hao, J.-K. (2017). An iterated "hyperplane exploration" approach for the quadratic knapsack problem. *Computers & Operations Research*, 77:226–239.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, 3rd edition.

Cunha, J. O., Simonetti, L., and Lucena, A. (2016). Lagrangian heuristics for the quadratic knapsack problem. *Computational Optimization and Applications*, 63:97–120.

Dantzig, G. B. (1957). Discrete-variable extremum problems. *Operations Research*, 5(2):266–277.

Fennich, M. E., Fomeni, F. D., and Coelho, L. C. (2024). A novel dynamic programming heuristic for the quadratic knapsack problem. *European Journal of Operational Research*, 319:102–120.

Fomeni, F. D., Kaparis, K., and Letchford, A. N. (2022). A cut-and-branch algorithm for the quadratic knapsack problem. *Discrete Optimization*, 44:100579.

Fomeni, F. D. and Letchford, A. N. (2014). A dynamic programming heuristic for the quadratic knapsack problem. *INFORMS Journal on Computing*, 26(1):173–182.

Gallo, G., Hammer, P. L., and Simeone, B. (1980). Quadratic knapsack problems. *Mathematical Programming Study*, 12:132–149.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York.

Kolesar, P. J. (1967). A branch and bound algorithm for the knapsack problem. *Management Science*, 13(9):723–735.

Leskovec, J., Rajaraman, A., and Ullman, J. D. (2020). *Mining of Massive Datasets*. Cambridge University Press, 3rd edition.

Ow, P. S. and Morton, T. E. (1988). Filtered beam search in scheduling. *International Journal of Production Research*, 26(1):35–62.

Peeters, R. (2025). Reader 9. Lecture notes on Combinatorial Optimization, Tilburg University. Accessed via Canvas.

Pisinger, D. (2007). The quadratic knapsack problem—a survey. *Discrete Applied Mathematics*, 155:623–648.

Pisinger, D., Rasmussen, A. B., and Sandvik, R. (2007). Solution of large-sized quadratic knapsack problems through aggressive reduction. Technical Report 2004/11, DIKU, University of Copenhagen, Copenhagen, Denmark.

Rodrigues, C. D., Quadri, D., Michelon, P., and Gueye, S. (2012). 0-1 quadratic knapsack problems: An exact approach based on a t-linearization. *SIAM Journal on Optimization*, 22(4):1449–1468.

Salkin, H. M. and de Kluyver, C. A. (1975). The knapsack problem: A survey. *Naval Research Logistics Quarterly*, 22(1):127–144.

Schauer, J. (2016). Asymptotic behavior of the quadratic knapsack problem. *European Journal of Operational Research*, 255:357–363.

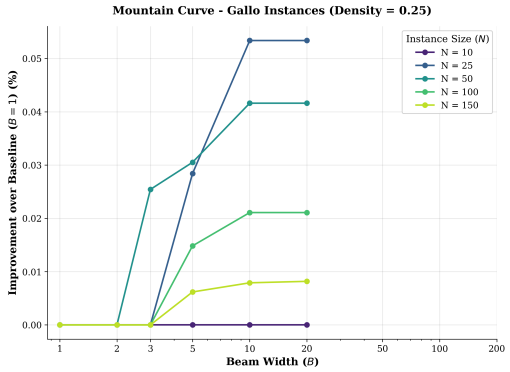
Sotirov, R. (2025). Chapter 6: The knapsack problem. Lecture notes on Combinatorial Optimization, Tilburg University. Accessed via Canvas.

Vershynin, R. (2018). *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press, 2nd edition.

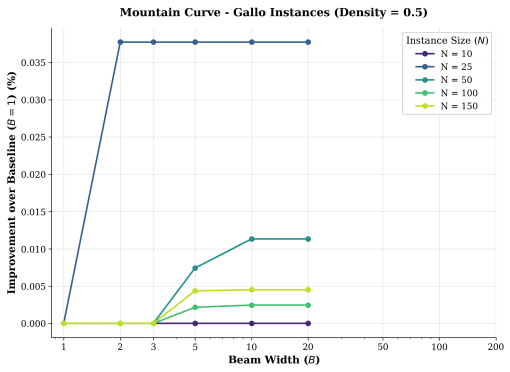
Appendix A

Extra Figures

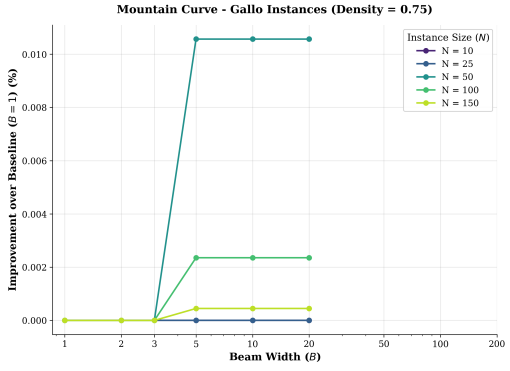
This appendix provides the detailed convergence plots for the standard benchmark instances, separated by edge density (d). These convergence curves illustrate the improvement of the Diverse Beam Search over the baseline ($B = 1$) across varying beam widths. In all tested configurations, the solutions reach within 99.9% of the best solution found at or before a beam width of $B = 5$, regardless of the underlying density.



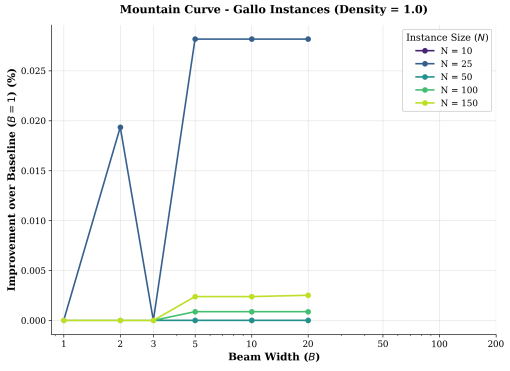
a) Gallo $d = 0.25$



b) Gallo $d = 0.50$

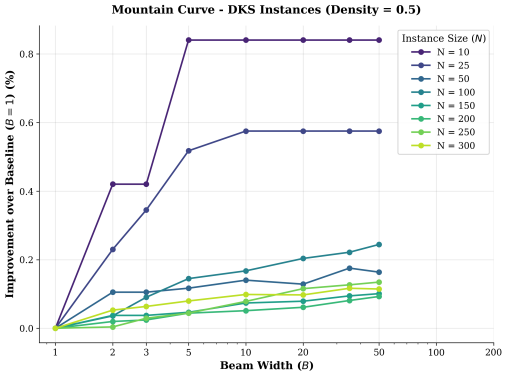


c) Gallo $d = 0.75$

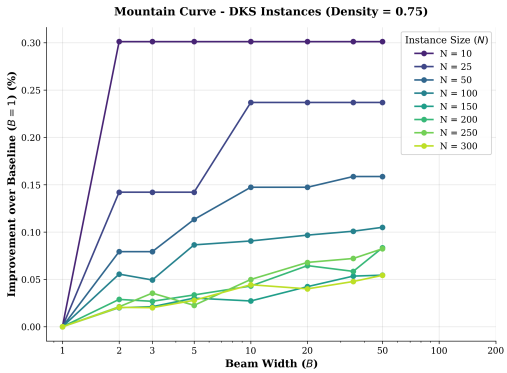


d) Gallo $d = 1.00$

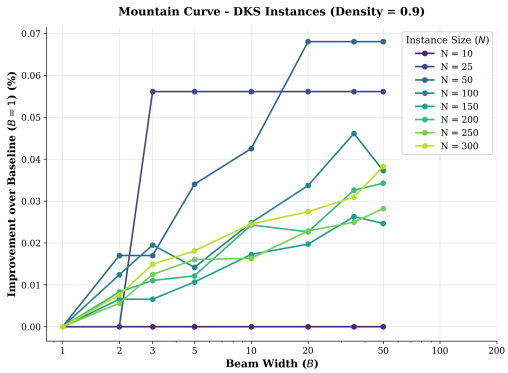
Figure A.1: Convergence curves for Gallo instances across all tested densities.



a) DKS $d = 0.50$



b) DKS $d = 0.75$



c) DKS $d = 0.90$

Figure A.2: Convergence curves for DKS instances across all tested densities.